ĐẠI HỌC BÁCH KHOA HÀ NỘI



PROJECT III

Kriky

Trợ lí hỗ trợ giáo viên ra đề và chữa bài

Giảng viên hướng dẫn: Vũ Văn Thiệu

Sinh viên thực hiện: Lê Nguyễn Phước Thành – 20235561

Mục lục

1 Đặt vấn đề **5**
- 1.1 Bối cảnh . . . 5
- 1.2 Vấn đề cần giải quyết . . . 7
- 1.3 Mục tiêu sản phẩm . . . 7
- 1.4 Phạm vi . . . 7

2 Ý tưởng sản phẩm **8**
- 2.1 Luồng tổng thể . . . 8
- 2.2 Hai giai đoạn làm bài của học sinh . . . 10
- 2.3 Làm sao phân tích lỗi của học sinh? . . . 10
- 2.4 Tạo câu hỏi mới dựa trên câu làm sai . . . 10
- 2.5 Thang điểm . . . 10
- 2.6 Giáo viên đưa ra quyết định ở đâu? . . . 11

3 Actor và khái niệm nghiệp vụ **12**
- 3.1 Hai actor . . . 12
  - 3.1.1 Teacher . . . 12
  - 3.1.2 Student . . . 12
- 3.2 Từ điển khái niệm . . . 12
  - 3.2.1 Người dùng, đề và bài làm . . . 12
  - 3.2.2 Thành phần hệ thống . . . 13
  - 3.2.3 Tài liệu . . . 14
  - 3.2.4 Trợ lí . . . 14

4 Các luồng nghiệp vụ **16**
- 4.1 Workflow 1 — Tạo và duyệt đề . . . 18
- 4.2 Workflow 2 — Làm bài (giai đoạn 1) . . . 18
- 4.3 Workflow 3 — Chấm bài . . . 18
- 4.4 Workflow 4 — Chữa bài (giai đoạn 2) . . . 19
  - 4.4.1 Giai đoạn 2 đếm giờ như thế nào? . . . 19
  - 4.4.2 Khi nào kết thúc giai đoạn 2 . . . 19
  - 4.4.3 Giáo viên làm gì? . . . 20
- 4.5 Business Workflows . . . 20

5 Use case **21**
- 5.1 Sáu use case . . . 21
- 5.2 Đặc tả từng use case . . . 23
  - 5.2.1 UC-01 — Tạo đề nháp . . . 23

5.2.2 UC-02 — Review và phát hành đề . . . 23
5.2.3 UC-03 — Làm và nộp bài . . . 23
5.2.4 UC-04 — Chữa bài . . . 23
5.2.5 UC-05 — Tạo câu biến thể . . . 24
5.2.6 UC-06 — Báo cáo Trợ lí giải thích chưa rõ . . . 24

6 Kiến trúc và mô hình dữ liệu 25
6.1 Các service trong hệ thống . . . 25
6.2 Frontend không nhìn thấy bên trong hệ thống . . . 27
6.3 Backend quyết định mọi thứ . . . 27
6.4 Service nào kết nối tới store nào? . . . 27
6.5 Các service nói chuyện với nhau thế nào . . . 28
6.6 Job nào đi qua queue, job nào không . . . 28
6.7 Nếu một service ngừng thì mất gì? . . . 29
6.8 Mô hình dữ liệu . . . 30

7 Chi tiết các service 33
7.1 `backend` . . . 33
7.1.1 Vào và ra . . . 33
7.1.2 Bảy mảng việc, và không có worker nào . . . 33
7.1.3 Ba cách lấy kết quả một job . . . 34
7.1.4 Các luồng đi qua `backend` . . . 35
7.1.5 Luật và chỗ canh luật . . . 37
7.1.6 Khi hỏng . . . 37
7.2 `agent` . . . 37
7.2.1 Vào và ra . . . 38
7.2.2 Bốn phần bên trong . . . 38
7.2.3 Một bước của lượt hội thoại: ba tầng phòng thủ . . . 39
7.2.4 Luật và chỗ canh luật . . . 40
7.2.5 Khi hỏng . . . 40
7.3 `document` . . . 41
7.3.1 Vào và ra . . . 41
7.3.2 Vì sao có hai worker . . . 42
7.3.3 Một tệp đi qua service . . . 42
7.3.4 Luật và chỗ canh luật . . . 44
7.3.5 Khi hỏng . . . 45

Tài liệu tham khảo 47

Danh mục hình vẽ

1.1 Domain Context 6

2.1 Tổng thể một vòng 9

4.1 Activity Overview 17
4.2 Business Workflows 20

5.1 Use Case Diagram 22

6.1 System Architecture 26

7.1 Một lượt hội thoại của giáo viên 36
7.2 Một tệp đi qua `document` 43

Danh mục bảng

2.1 Ba mức điểm của một câu hỏi 11

5.1 Sáu use case và actor chính của từng cái 21

6.1 Năm service của hệ thống 25
6.2 Bảy task gọi Model Provider 29
6.3 Mỗi service ngừng thì mất gì, còn gì 30
6.4 Hai mươi hai bảng dữ liệu 30

7.1 Bảy mảng việc của `backend` 34
7.2 Ba cách `backend` lấy kết quả một job 34
7.3 Luật `backend` giữ 37
7.4 Khi một thao tác của `backend` hỏng 38
7.5 Sáu task của `agent` 39
7.6 Luật `agent` giữ 40
7.7 Khi một thao tác của `agent` hỏng 41
7.8 Sáu collection của content store 42
7.9 Bảy khâu dựng cây mục lục 44
7.10 Luật `document` giữ 45
7.11 Khi một thao tác của `document` hỏng 46

Chương 1

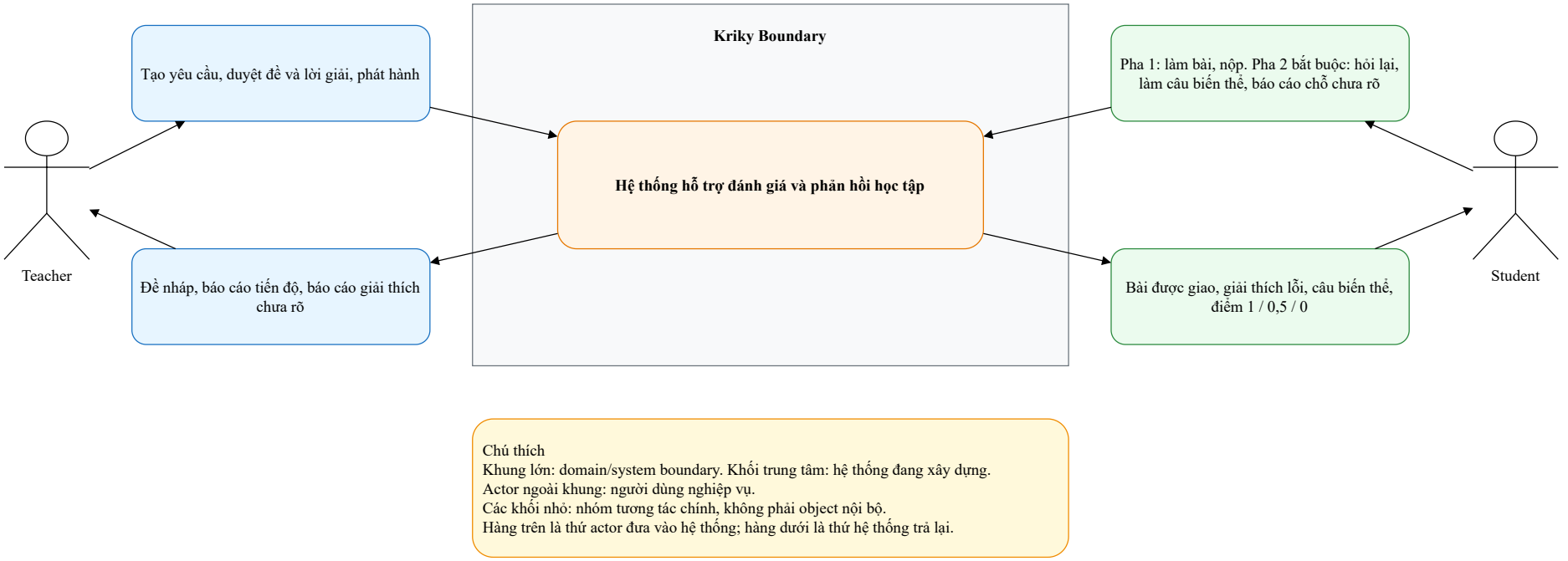
Đặt vấn đề

1.1 Bối cảnh

Ở trường phổ thông, một bài kiểm tra thường kết thúc khi giáo viên trả điểm. Học sinh nhận lại bài, nhìn những câu bị gạch chéo, rồi phần lớn dừng ở đó. Thứ thực sự làm thay đổi kết quả học tập là việc hiểu ra mình sai ở đâu và làm lại được, nhưng việc đó hiếm khi xảy ra, vì nó cần một cuộc trao đổi riêng cho từng em về từng câu.

Giáo viên không phải không muốn làm. Vấn đề nằm ở khối lượng công việc. Một lớp bốn mươi học sinh làm một đề mười câu sẽ sinh ra bốn trăm lượt trả lời, và mỗi lượt sai có thể sai vì một lý do khác nhau. Giải thích riêng cho từng em, rồi ra thêm một câu tương tự để kiểm xem em đã sửa được chưa, là công việc tăng theo sĩ số và không có cách nào rút gọn.

Kriky là hệ thống được xây để gánh phần công việc đó, nhưng không lấy đi quyền quyết định của giáo viên. Hình 1.1 cho thấy ranh giới của hệ thống: hai **actor** (tác nhân, mục 3.2) ở hai phía, cùng những gì đi vào và đi ra.



Hình 1.1: Domain Context — ranh giới hệ thống, hai actor, và những gì đi qua ranh giới ấy

1.2 Vấn đề cần giải quyết

Phần tốn thời gian nhất của giáo viên không phải là gõ ra đề bài, mà là năm việc sau. Cả năm đều là công việc chuyên môn:

- Soạn đề bám đúng mục tiêu học tập, thay vì gom câu hỏi cho đủ số lượng.
- Tạo phương án nhiễu có ý nghĩa. Một phương án sai ngẫu nhiên không nói lên điều gì, còn một phương án nhiễu tốt là phương án mà học sinh mắc một lỗi cụ thể sẽ chọn.
- Chấm bài và viết nhận xét cho từng học sinh.
- Phân định học sinh sai vì thiếu kiến thức, vì suy luận hỏng, vì tính nhầm, hay vì hiểu sai một khái niệm.
- Ra câu biến thể của đúng câu học sinh vừa làm sai, để kiểm xem em đã sửa được chỗ sai đó hay chỉ nhớ được đáp án.

Phía học sinh, nhu cầu ngược lại nhưng khớp vào: phản hồi cần đến nhanh, và bài luyện tập cần đúng với điểm yếu của riêng mình. Một phản hồi đúng nhưng đến sau hai tuần thì không còn tác dụng, vì học sinh đã quên mình nghĩ gì lúc làm bài.

1.3 Mục tiêu sản phẩm

Hệ thống hướng tới ba mục tiêu nghiệp vụ:

1. Hỗ trợ giáo viên tạo đề từ một yêu cầu bằng ngôn ngữ tự nhiên. Một đề có hai nguồn câu hỏi: nội dung sinh mới, và **Question Bank** (ngân hàng câu hỏi, mục 3.2).
2. Phân tích lỗi sai hoặc hiểu nhầm khái niệm của học sinh thông qua cuộc trò chuyện thay vì một câu nhận xét.
3. Tạo các câu hỏi mới từ các câu mà học sinh làm sai ngay trong cùng bài kiểm tra.

1.4 Phạm vi

Hệ thống dựng trọn một vòng nghiệp vụ, từ đầu đến cuối:

- Giáo viên tạo, xem lại và phát hành đề. Việc duyệt bao gồm cả lời giải và ánh xạ từ phương án nhiễu sang lỗi, không chỉ câu hỏi và đáp án.
- Giai đoạn 1: Học sinh làm bài, nộp bài, và hệ thống chấm bài để biết học sinh đang thiếu kiến thức ở phần nào.
- Giai đoạn 2: Học sinh trò chuyện với trợ lí Kriky để hiểu cách làm bài và tại sao lại sai. Sau đó hệ thống tạo câu hỏi mới, học sinh làm lại cho tới khi hiểu được cách làm của dạng bài.

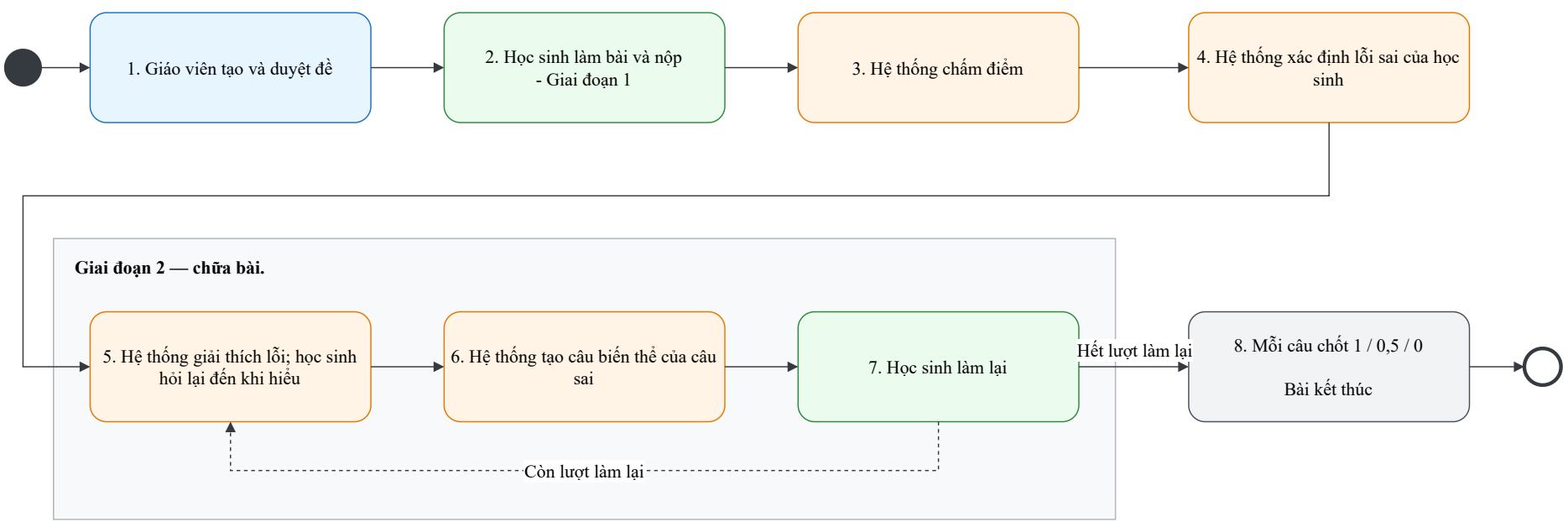
Chương 2

Ý tưởng sản phẩm

2.1 Luồng tổng thể

Ý tưởng lõi không nằm ở việc sinh câu hỏi, mà ở việc khép kín một vòng tám bước:

1. Giáo viên tạo và duyệt đề.
2. Học sinh làm bài.
3. Hệ thống chấm điểm.
4. Hệ thống xác định lỗi sai của học sinh.
5. Hệ thống giải thích lỗi, học sinh hỏi lại đến khi hiểu.
6. Hệ thống tạo câu biến thể của câu sai.
7. Học sinh làm lại.
8. Mỗi câu chốt ở một trong ba mức điểm, và bài kết thúc.



Hình 2.1: Tổng thể một vòng

2.2 Hai giai đoạn làm bài của học sinh

Bước 2, 7 và 8 của hình 2.1.

Một bài kiểm tra có hai giai đoạn: giai đoạn 1 là làm bài và nộp, giai đoạn 2 là chữa những câu đã sai. Nộp bài kết thúc giai đoạn 1 chứ không kết thúc bài. Bài chỉ kết thúc khi giai đoạn 2 xong, hoặc khi hạn của giai đoạn 2 hết.

Kéo theo đó là một hệ quả cần nói rõ: một bài kiểm tra có hai trạng thái hoàn thành khác nhau, *đã nộp* và *đã hoàn thành*, và chúng không thay thế cho nhau được.

2.3 Làm sao phân tích lỗi của học sinh?

Bước 4 của hình 2.1.

Với hình thức trắc nghiệm, việc chấm chỉ là một phép so. Khó khăn nằm ở chẩn đoán lỗi sai của học sinh, bởi cùng chọn một phương án sai có thể do nhiều lỗi khác nhau sinh ra. Nếu hệ thống đoán lỗi rồi đi giải thích một lỗi học sinh không hề mắc thì nó còn tệ hơn im lặng: em vừa mất thời gian, vừa được dạy sai chỗ.

Vì vậy, mỗi phương án sai phải mang theo lỗi mà nó đại diện, và được soạn cùng câu hỏi chứ không suy ra lúc chấm.

Hai luật đi kèm:

- Mỗi câu hỏi phải mang theo lời giải có nhiều hơn một cách làm.
- Số phương án không cố định. Một câu có thể có ba, bốn hay năm lựa chọn tuỳ từng câu, nhưng chỉ đúng một lựa chọn là đáp án đúng. Mọi phương án còn lại đều là nhiễu, nên đều phải có lỗi gắn kèm.

2.4 Tạo câu hỏi mới dựa trên câu làm sai

Bước 6 và 7 của hình 2.1

Khi học sinh sai câu 3, hệ thống sinh câu 3̀, gọi là câu biến thể. Thứ cần chữa là một lỗi cụ thể vừa xảy ra, không phải một chủ đề.

Câu biến thể giữ nguyên dạng đề và cách làm, chỉ đổi dữ kiện, con số và cách hỏi. Đó là cách duy nhất kiểm được rằng học sinh đã sửa đúng chỗ mình sai, chứ không phải chỉ nhớ được đáp án.

2.5 Thang điểm

Bước 8 của hình 2.1.

Ba mức thay vì hai, vì hai mức sẽ biến việc chữa bài thành công việc không được có ý nghĩa. Một học sinh sửa được lỗi của mình đã học đúng thứ mà bài kiểm tra muốn dạy, và mức giữa là chỗ duy nhất ghi nhận điều đó. Vì vậy giai đoạn 2 chỉ cộng chứ không bao giờ trừ: điểm tính ngay sau khi nộp là sàn.

Bảng 2.1: Ba mức điểm của một câu hỏi

Điểm	Nghĩa
1	Đúng ngay ở giai đoạn 1
0,5	Sai ở giai đoạn 1 nhưng chữa được ở giai đoạn 2
0	Sai và không chữa được

2.6 Giáo viên đưa ra quyết định ở đâu?

Bước 1 của hình 2.1.

Với nội dung đề, trợ lí viết còn giáo viên quyết nội dung đó có đến tay học sinh hay không. Giáo viên bắt buộc tham gia ở hai điểm, một ở đầu ra và một ở đầu vào:

1. Trợ lí hỏi lại khi chưa đủ thông tin, thay vì tự đoán (đầu vào).
2. Duyệt đề trước khi phát hành (đầu ra).

Chương 3

Actor và khái niệm nghiệp vụ

3.1 Hai actor

Hệ thống có đúng hai **actor** (tác nhân) nghiệp vụ: **Teacher** (giáo viên) và **Student** (học sinh).

3.1.1 Teacher

Teacher là người đưa ra các quyết định. Teacher có thể:

- Nhập yêu cầu tạo đề.
- Xem và chỉnh sửa đề do trợ lí tạo.
- Duyệt đề trước khi phát hành.
- Phát hành đề cho một hoặc nhiều lớp.
- Xem kết quả làm bài của học sinh.
- Xem báo cáo học sinh gửi về chỗ trợ lí giải thích chưa rõ.

3.1.2 Student

Student là người nhận đề, làm bài và luyện tập. Student có thể:

- Xem bài kiểm tra được giao.
- Chọn đáp án và nộp bài (giai đoạn 1).
- Nghe giải thích lỗi và hỏi lại đến khi hiểu (giai đoạn 2).
- Làm câu biến thể của những câu đã sai (giai đoạn 2).
- Báo cáo chỗ trợ lí giải thích chưa rõ.

3.2 Từ điển khái niệm

3.2.1 Người dùng, đề và bài làm

- **Actor**: Hệ thống có hai actor: *Teacher* là giáo viên, *Student* là học sinh. Báo cáo chỉ dùng hai chữ tiếng Anh này khi nói về actor và trong sơ đồ; khi kể thì gọi là giáo viên, học sinh.
- **Use case**: Một chức năng hệ thống mang lại cho actor.
- **Workflow**: Luồng nghiệp vụ, tức chuỗi hoạt động mà các actor và hệ thống lần lượt làm để đạt một mục tiêu, như tạo và duyệt một đề. Một workflow có thể gồm nhiều use case.

- **Assessment**: Bài kiểm tra, hay bộ câu hỏi được giao cho học sinh.
- **Question**: Câu hỏi trong một assessment.
- **Distractor**: Phương án sai *có chủ đích*, đại diện cho một lỗi hoặc một hiểu nhầm cụ thể. Không phải đáp án nhiễu ngẫu nhiên.
- **Lời giải**: Cách làm đi kèm mỗi câu hỏi, bắt buộc có nhiều hơn một cách. Trợ lí dựa vào lời giải để giải thích cho học sinh.
- **Draft**: Đề đang soạn, chưa duyệt. Câu hỏi ở trạng thái này sửa và xoá được tự do.
- **Approve**: Chốt rằng đề đã dùng được. Đây là một *trạng thái*, không phải một thao tác: nó có cả chiều ngược, gọi là *unapprove*, và chỉ đề đã approve mới phát hành được.
- **Question Bank**: Ngân hàng câu hỏi riêng của từng giáo viên.
- **Class**: Lớp học có sẵn danh sách học sinh. Giáo viên tạo lớp. Phát hành đề phải chọn một lớp đã tạo từ trước.
- **Publish** và **Publication**: *Publish* là phát hành một đề cho một lớp. Một *Publication* là bản ghi sinh ra từ việc đó.
- **Giai đoạn 1**: Phần làm bài và nộp.
- **Giai đoạn 2**: Phần chữa bài.
- **opens_at** / **closes_at** / **remediation_deadline**: Ba mốc thời gian của một *Publication*. Hai mốc đầu là thời điểm mở và đóng giai đoạn 1; mốc thứ ba là thời điểm đóng giai đoạn 2.
- **Submission**: Việc nộp bài, và là mốc kết thúc *giai đoạn 1* — không phải mốc kết thúc bài kiểm tra.
- **Attempt**: Một lần làm bài của một học sinh trên một đề, trải qua **cả hai giai đoạn**. Mỗi học sinh có duy nhất một Attempt cho mỗi đề — em chưa vào làm thì không có Attempt nào. Nó xong khi giai đoạn 2 hoàn thành.
- **Mark** và **Score**: *Mark* là điểm của *một câu*; *score* là điểm của *cả bài*.
- **Misconception**: Hiểu nhầm khái niệm, hoặc lỗi sai có tính lặp lại.
- **Feedback**: Phản hồi / nhận xét của trợ lí cho những thắc mắc của học sinh.
- **Lượt**: Một lượt cho phép học sinh làm lại các câu sai của giai đoạn trước / lượt trước. Khác với một *lượt hội thoại*: một tin nhắn cùng mọi thứ trợ lí làm để đáp lại tin ấy.
- **Câu biến thể**: Câu sinh từ *câu học sinh làm sai ở giai đoạn 1*: giữ cấu trúc và lỗi cần kiểm, chỉ đổi dữ kiện.

3.2.2 Thành phần hệ thống

- **Service**: Một đơn vị chạy độc lập trong hệ thống. Hiện tại có năm service: `frontend`, `backend`, `agent`, `document`, `ingest`.
- **API** (Application Programming Interface): Giao diện lập trình, tức cách một chương trình gọi một chương trình khác. Giữa các service, chỉ `frontend` gọi `backend` theo cách này; các service nội bộ không gọi API để giao tiếp với nhau.
- **Gateway**: `backend` là gateway của cả hệ thống: không màn hình nào của `frontend` gọi thẳng `agent`, `document` hay `ingest`.
- **VPC** và **Subnet**: *VPC* là đám mây riêng ảo, tức một vùng mạng riêng tách khỏi bên ngoài; nó chứa mọi service trừ `frontend`. Nó chia thành các *subnet*, tức các vùng mạng con: chỉ những gì nằm ở *public subnet* (vùng mạng công khai) mới nhận được kết nối từ bên ngoài, còn những gì nằm ở *private subnet* (vùng mạng nội bộ) thì không. Hình kiến trúc ở chương 6 đặt `backend` ở public subnet, mọi service và kho khác ở private subnet.
- **Job** và **Task**: Hai thứ khác nhau. Một *task* là *tên* của một loại việc. Một *job* là một lần chạy của task ấy, mỗi job có một mã riêng.
- **Queue**: Nơi một service đẩy job vào và một service khác lấy ra làm. Job nằm lại trên queue

đến khi có ai lấy, nên tắt máy giữa chừng không mất việc.
- **Worker**: Tiến trình ngồi nghe một queue, lấy job xuống và chạy. Nó không nhận lời gọi từ ngoài; thứ duy nhất nó đọc là queue. Một service có thể có nhiều worker: `document` có một worker đọc chữ của tệp và một worker dựng cây mục lục.
- **Byte**: Đơn vị cơ bản để đo và lưu dữ liệu trong máy; mọi tệp, dù là PDF hay ảnh chụp, đều là một dãy byte. Báo cáo nói *byte của tệp*, tức dữ liệu gốc, để chỉ chính tệp giáo viên tải lên, nguyên như lúc tải, khác với chữ hệ thống đọc ra từ nó.
- **Store**: Kho dữ liệu mà service đọc và ghi. Hệ thống có bốn store thuộc bốn loại khác nhau, nên mỗi loại mang chữ riêng: *database* giữ dữ liệu có cấu trúc, *object store* giữ byte của tệp, *content store* giữ nội dung đã xử lí của tài liệu, như chữ từng trang và cây mục lục, và *queue* giữ job đang chờ.
- **Collection**: Nhóm bản ghi cùng loại trong content store, giống một bảng của database.
- **Credential**: Khoá cho phép một service vào một store để đọc / ghi.
- **Pub/sub** và **Channel**: *Pub/sub* là cách một service phát tin lên một *channel*, tức một kênh có tên: ai đang nghe channel ấy thì nhận, ai không nghe thì mất. Khác queue ở chỗ nó **không giữ lịch sử**, nên hệ thống chỉ phát qua nó những thứ mất được.
- **SSE** và **Polling**: Hai cách để màn hình cập nhật thời gian thực (realtime). Với *SSE*, `backend` giữ một kết nối mở và gửi tin xuống qua đó mỗi khi có gì đổi. Với *polling*, màn hình tự hỏi lại theo một chu kì. Hệ thống hiện tại dùng *SSE*.
- **Event**: Một tin `backend` gửi xuống màn hình qua SSE.

3.2.3 Tài liệu

Nhóm này đi theo một tài liệu, từ tệp giáo viên tải lên tới những mẩu chữ trợ lí đọc:

- **Document**: Tài liệu giáo viên tải lên, là một tệp PDF hoặc tệp chữ thuần `.txt`, `.md`. Nó nằm trong kho lưu trữ riêng của từng tài khoản, cung cấp kiến thức và giới hạn phạm vi ra đề. Một tài liệu chỉ dùng được khi hệ thống đã đọc ra chữ của nó, kể cả đọc từ ảnh nếu tệp là bản chụp, và đã dựng xong *cây mục lục*.
- **OCR** (Optical Character Recognition): Đọc chữ từ ảnh trong tài liệu.
- **Bản thô**: Những gì `document` đọc được từ một tệp, chia theo trang và theo khối chữ, trước khi có cây mục lục. Bản thô được ghi vào content store trước, rồi cây mục lục mới được dựng từ nó.
- **Cây mục lục**: Mục lục của một *Document* mà hệ thống tự dựng từ nội dung của tệp. Cây có tối đa ba cấp, ví dụ: chương, bài và mục.
- **Chunk**: Một mẩu chữ ngắn cắt từ bản thô, thường thuộc về đúng một nút của cây mục lục. Khi trợ lí cần đọc một bài của tài liệu, `backend` lấy các chunk của bài ấy và các mục bên dưới đưa cho nó, chứ không đưa cả tài liệu.
- **Scope**: Phạm vi ra đề trong một *Document*: các nút giáo viên chọn trên cây mục lục, như một chương hay vài bài, cùng mọi mục nằm dưới chúng. Scope trỏ vào nút, không trỏ vào số trang.

3.2.4 Trợ lí

- **Model Provider**: Nhà cung cấp mô hình ngôn ngữ, nằm ngoài hệ thống. Chỉ hai service gọi tới nó: `agent`, để soạn câu và trò chuyện, và `document`, khi dựng cây mục lục.
- **Tool**: Một việc có tên mà `agent` được phép *đề xuất* chạy, kèm tham số. Có tool tra cứu, như xem lớp, xem kết quả của một đề, đọc mục lục và chữ của tài liệu; và tool ghi, như mở đề, soạn câu, thay một câu. Danh sách tool là một **ràng buộc**, không phải một tiện ích:

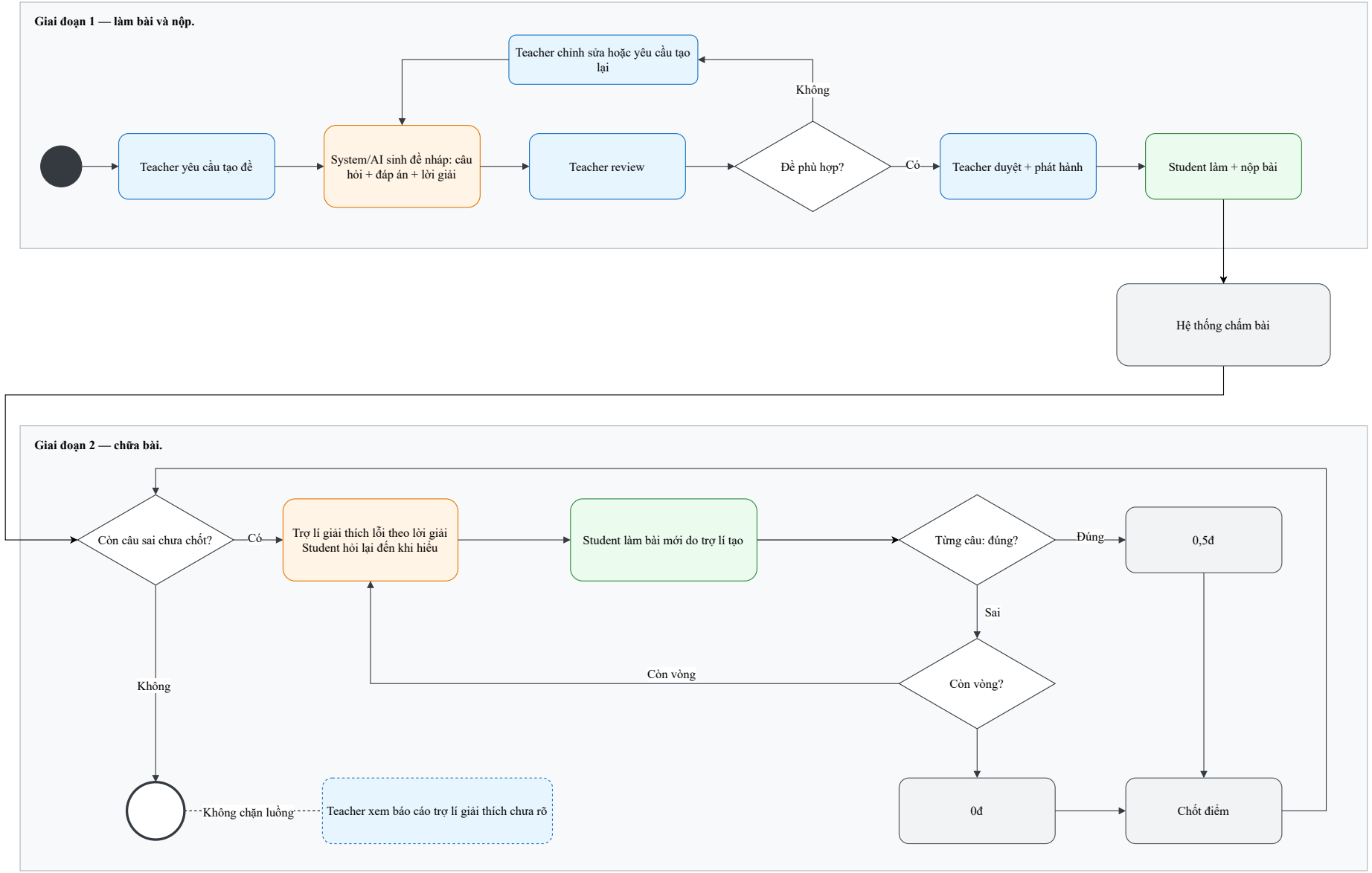
duyệt đề và phát hành đề cố ý không có tool nào, nên `agent` không có đường nào để đề xuất chúng.

- **Dàn ý**: Kế hoạch của một đề mà trợ lí đưa ra *trước* khi soạn câu nào. Mỗi mục của dàn ý nói mục ấy có mấy câu, kiểm kiến thức gì, dùng công thức nào, làm bài theo cách nào, và dựa vào bài nào của tài liệu nếu có. Giáo viên đồng ý thì câu hỏi mới được soạn.
- **Plan**: Danh sách việc trợ lí định làm cho một tin nhắn của giáo viên, mỗi việc là một *tool* kèm tham số.
- **Graph**: Cách `agent` và `document` tổ chức một job có gọi mô hình: một chuỗi các hàm cố định, dựng bằng thư viện LangGraph. Mỗi hàm làm một phần rồi trao kết quả cho hàm sau; chuỗi có thể quay lại một hàm trước, như khi hàm viết câu hỏi không qua được hàm kiểm tra và phải viết lại.

Chương 4

Các luồng nghiệp vụ

Nghiệp vụ của hệ thống gồm bốn luồng. Hình 4.1 và hình 2.1 mô tả cùng một luồng, nhưng ở hai mức chi tiết khác nhau.



Hình 4.1: Activity Overview — toàn bộ vòng, hai giai đoạn nằm trong hai khung riêng

4.1 Workflow 1 — Tạo và duyệt đề

Mục tiêu. Giúp giáo viên tạo được đề phù hợp với mục tiêu học tập, nhưng giáo viên vẫn là người duyệt cuối cùng trước khi phát hành.

Luồng chính.

1. **Teacher** (giáo viên) nhập yêu cầu tạo đề.
2. Teacher cung cấp ràng buộc: chủ đề, độ khó, số lượng câu hỏi, loại bài kiểm tra, thời gian làm bài.
3. `System/AI` phân tích yêu cầu.
4. `System/AI` tạo đề nháp gồm câu hỏi, phương án trả lời, đáp án đúng, lời giải và **metadata** (thông tin mô tả) cơ bản.
5. Teacher xem đề nháp.
6. Teacher chỉnh sửa thủ công, hoặc yêu cầu tạo lại một phần.
7. Teacher duyệt đề.
8. Teacher phát hành đề cho **Student** (học sinh).

Điểm kiểm soát. Teacher phải duyệt đề trước khi đề được phát hành. Đề kiểm tra ảnh hưởng trực tiếp đến việc đánh giá học sinh, nên trợ lí không được tự phát hành.

4.2 Workflow 2 — Làm bài (giai đoạn 1)

Mục tiêu. Cho phép học sinh làm bài và ghi nhận bài nộp để hệ thống đánh giá.

Luồng chính.

1. Student nhận **assessment** (bài kiểm tra) đã được Teacher phát hành.
2. Student vào làm, *tới hết* giờ đóng.
3. Student trả lời câu hỏi trắc nghiệm.
4. Student nộp bài.
5. Hệ thống ghi nhận **submission** (việc nộp bài) để đánh giá.

Ba tham số thời gian. Giai đoạn 1 có ba tham số thời gian độc lập: thời gian làm bài (`phase1_minutes`), giờ mở (`opens_at`) và giờ đóng (`closes_at`).

Luật là: giờ đóng chặn việc *vào làm*, không chặn việc nộp. Học sinh vào tới hết giờ đóng vẫn được. Ai đã vào rồi thì không bị dừng giữa chừng, và được đủ thời gian làm bài kể cả khi tràn qua giờ đóng. Ví dụ, đóng lúc 18:00 với thời gian làm bài 15 phút thì bài cuối có thể nộp lúc 18:15.

4.3 Workflow 3 — Chấm bài

Mục tiêu. Biết được những câu mà học sinh làm sai, từ đó trả lời các thắc mắc của học sinh thông qua cuộc trò chuyện và tạo ra các câu hỏi biến thể để kiểm tra độ hiểu bài.

Luồng chính.

1. Học sinh nộp bài.
2. Hệ thống **so** phương án đã chọn với đáp án đúng của câu hỏi đã duyệt. Đây là một phép so, không phải một phán đoán — nên không có gì để mà không chắc.
3. Điểm giai đoạn 1 chốt ngay và học sinh thấy ngay.

Hai điều kiện phải có sẵn từ lúc duyệt đề.

- Mỗi phương án nhiễu phải đại diện cho một lỗi cụ thể, soạn cùng lúc với câu hỏi. Phương án nhiễu mà không đại diện cho một lỗi cụ thể thì không giúp học sinh tiến bộ.
- Mỗi câu hỏi phải kèm lời giải. Việc chấm không dùng tới nó, nhưng phương án nhiễu phải được tạo dựa trên lời giải thì mới có ý nghĩa và trợ lí phải dựa vào lời giải để giải thích cho học sinh chứ không tự nghĩ ra cách giải mới. Chính vì thế lời giải phải tồn tại từ lúc duyệt đề chứ không sinh ra lúc học sinh hỏi.

4.4 Workflow 4 — Chữa bài (giai đoạn 2)

Mục tiêu. Bài kiểm tra chỉ kết thúc khi học sinh hiểu bài và sửa được lỗi họ mắc phải.

Luồng chính.

1. Học sinh trò chuyện với trợ lí Kriky về lỗi sai, chỗ nào chưa hiểu, cách giải như thế nào.
2. Kriky dựa vào *lời giải đã soạn kèm câu hỏi*. Học sinh hỏi lại đến khi hiểu. Phần này **không tính giờ**.
3. Trong lúc đó, hệ thống sinh câu biến thể của câu mà học sinh làm sai.
4. Học sinh bấm nút làm bài mới. Đồng hồ của lượt bắt đầu chạy.
5. Học sinh làm các câu mới trong lượt.
6. Câu nào đúng thì chốt **0,5** điểm; câu nào còn sai thì sang vòng tiếp.
7. Mỗi câu tối đa **ba vòng**. Hết ba vòng mà vẫn sai thì chốt **0** điểm.
8. Bài kết thúc khi mọi câu đã chốt, hoặc khi hết hạn giai đoạn 2.

4.4.1 Giai đoạn 2 đếm giờ như thế nào?

Giai đoạn 2 nhận hai tham số: số phút mỗi câu (`phase2_minutes_per_question`), và hạn kết thúc giai đoạn 2 (`remediation_deadline`).

Thời lượng của một lượt bằng số phút mỗi câu nhân với số câu còn sai trong lượt đó, nên sai hai câu với thời lượng làm lại 5 phút mỗi câu thì lượt ấy dài 10 phút.

Phần giải thích không tính giờ. Đồng hồ chỉ bắt đầu khi học sinh tự bấm nút làm bài mới.

4.4.2 Khi nào kết thúc giai đoạn 2

Hạn kết thúc giai đoạn 2 là một mốc tuyệt đối do giáo viên đặt, chung cho cả lớp, và dài hơn hẳn hạn của giai đoạn 1, chẳng hạn là cuối ngày.

Hết hạn thì dừng, kể cả khi học sinh đang làm dở một lượt, và câu chưa xong nhận 0 điểm.

Đây là chỗ giai đoạn 2 làm ngược với giai đoạn 1. Ở giai đoạn 1, học sinh đã vào thì không bị dừng giữa chừng; ở giai đoạn 2, hết hạn là dừng. Hai luật nghịch nhau một cách có chủ ý:

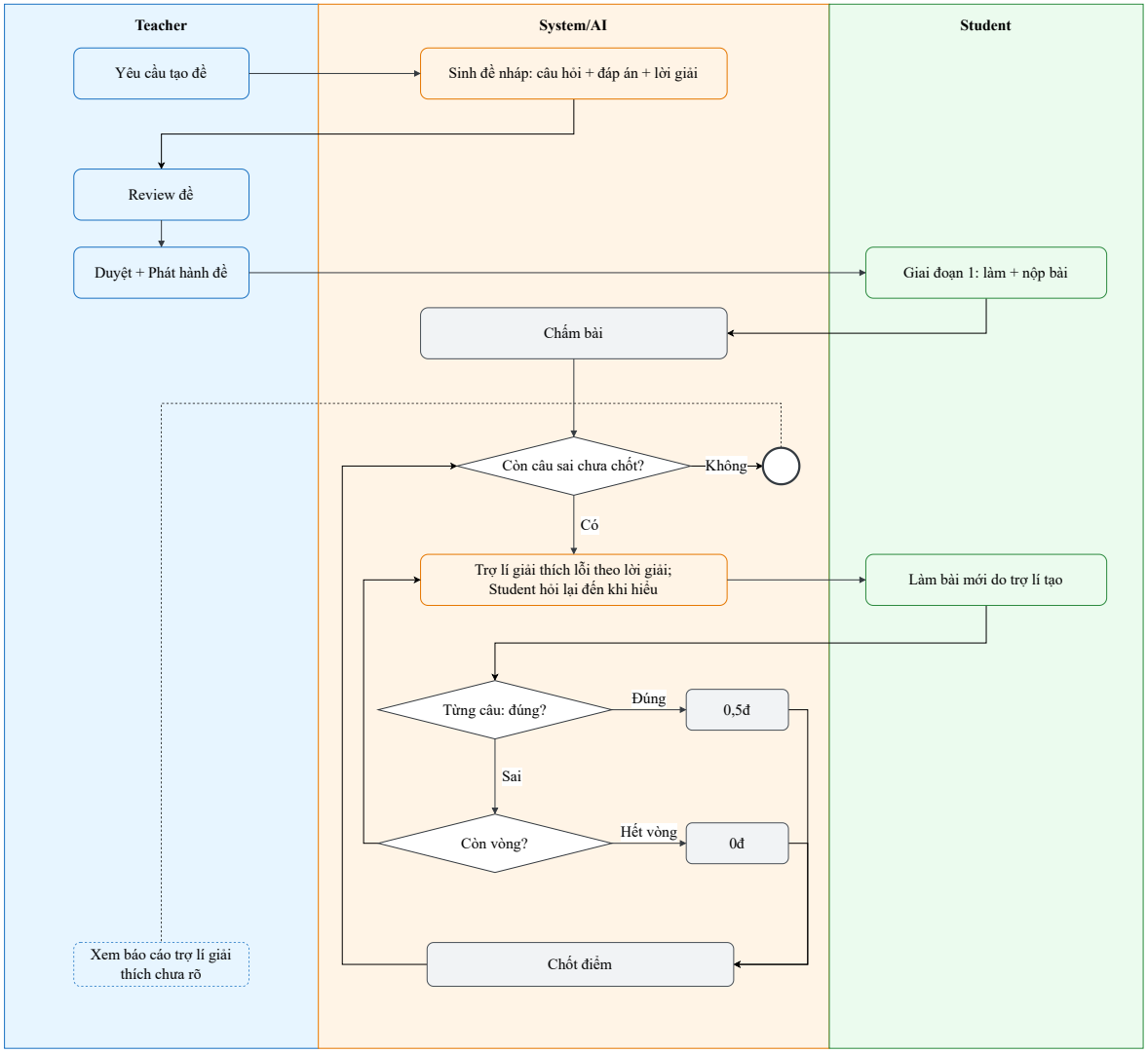
thời gian ở giai đoạn 1 là một lượng hệ thống cấp cho từng người kể từ lúc em vào, còn hạn của giai đoạn 2 là một mốc chung cả lớp nhìn thấy suốt ngày — không cắt thì mốc ấy không tồn tại.

4.4.3 Giáo viên làm gì?

Học sinh báo cáo một câu hoặc một đoạn hội thoại là *giải thích chưa rõ*. Giáo viên xem được các báo cáo này và giải thích để đưa vào bài dạy trên lớp.

4.5 Business Workflows

Hình 4.1 cho thấy luồng tổng quan; hình 4.2 cho thấy trách nhiệm của từng vị trí.



Hình 4.2: Business Workflows

Chương 5

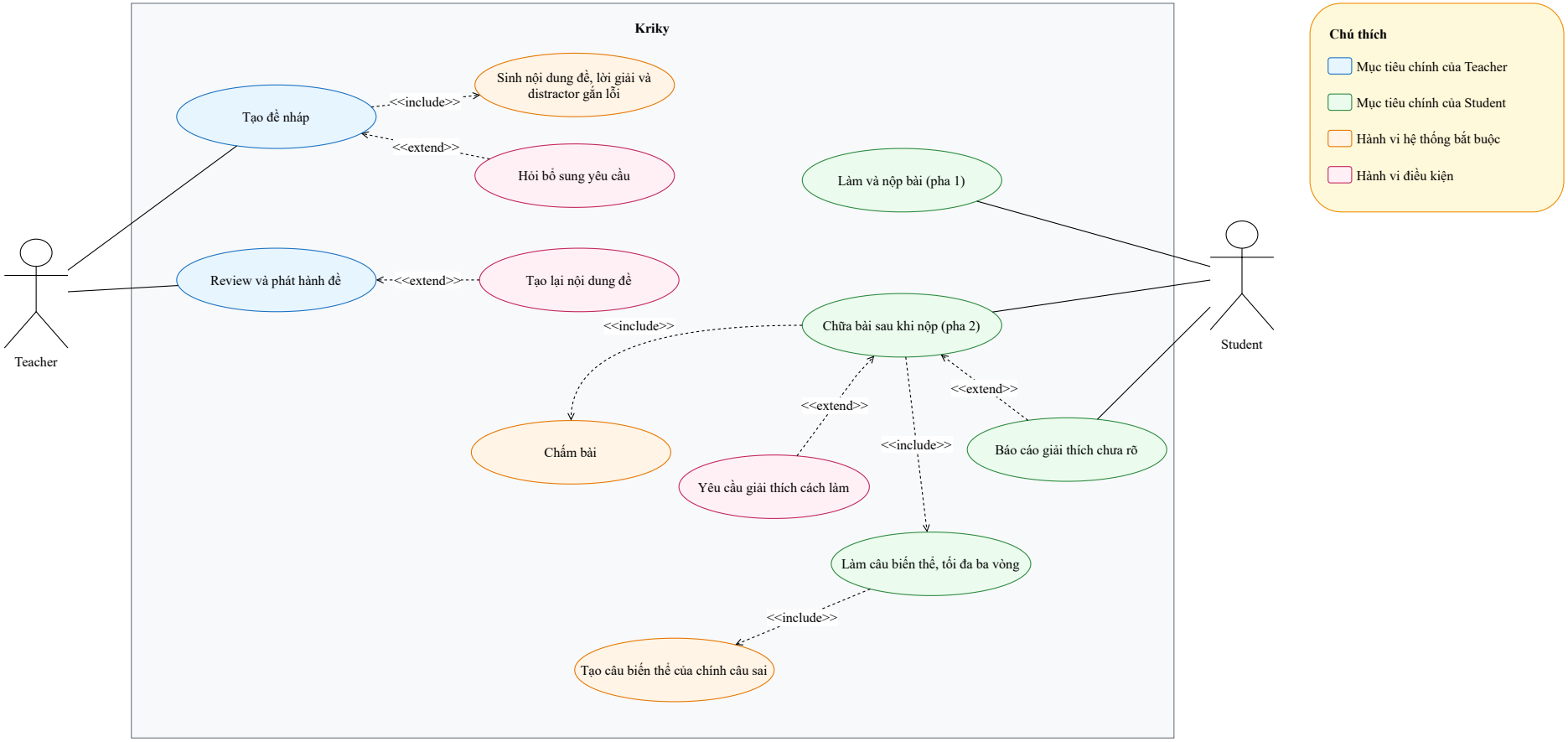
Use case

5.1 Sáu use case

Bảng 5.1: Sáu use case và actor chính của từng cái

Mã	Actor chính	Tên
UC-01	Teacher	Tạo đề nháp
UC-02	Teacher	Review và phát hành đề
UC-03	Student	Làm và nộp bài (giai đoạn 1)
UC-04	Student	Chữa bài sau khi nộp (giai đoạn 2)
UC-05	—	Hệ thống tạo câu biến thể
UC-06	Student	Báo cáo chỗ trợ lí giải thích chưa rõ

UC-05 không có **actor** (tác nhân) trực tiếp: đó là hành vi hệ thống, được `<<include>>` từ UC-04.



Hình 5.1: Use Case Diagram

5.2 Đặc tả từng use case

5.2.1 UC-01 — Tạo đề nháp

Điều kiện trước. **Teacher** (giáo viên) biết chủ đề hoặc mục tiêu học tập cần kiểm tra, và mô tả được yêu cầu ở mức đủ hiểu.

Luồng chính. Teacher nhập yêu cầu và các ràng buộc (chủ đề, số câu, độ khó, thời gian làm bài, loại bài). Hệ thống phân tích, sinh đề nháp gồm câu hỏi, các phương án, đáp án đúng, và lời giải rồi hiển thị cho Teacher.

Quan hệ. *Tạo đề nháp* `<<include>>` *Sinh nội dung đề*. *Hỏi bổ sung yêu cầu* `<<extend>>` *Tạo đề nháp*, khi yêu cầu ban đầu chưa đủ rõ — đây chính là một trong hai nơi Giáo viên ra quyết định ở mục 2.6.

5.2.2 UC-02 — Review và phát hành đề

Điều kiện trước. Đã có đề nháp.

Luồng chính. Teacher xem từng câu hỏi, xem đáp án đúng, lời giải và phương án sai; chỉnh sửa nếu cần; duyệt đề; rồi phát hành với sáu tham số. Mỗi lần phát hành cho một lớp sinh ra một **Publication** (bản ghi phát hành).

Quan hệ. *Tạo lại nội dung đề* `<<extend>>` *Review và phát hành đề*, khi Teacher thấy một phần hoặc toàn bộ đề chưa phù hợp.

5.2.3 UC-03 — Làm và nộp bài

Điều kiện trước. Bài đã được phát hành và **Student** (học sinh) có quyền truy cập.

Luồng chính. Student mở bài, đọc câu hỏi, chọn đáp án, nộp bài. Hệ thống ghi nhận, và chấm bài.

Sơ đồ *không* dùng `<<include>>` từ *Làm và nộp bài* sang *Chấm bài*: chấm bài là bước xử lý *sau* khi có bài nộp, không phải hành vi bắt buộc để Student hoàn thành mục tiêu nộp bài của mình.

Kết quả. Bài nộp được ghi nhận và được chấm. **Nộp bài không kết thúc bài kiểm tra** — nó kết thúc giai đoạn 1.

5.2.4 UC-04 — Chữa bài

Điều kiện trước. Student đã nộp bài ở giai đoạn 1 và hệ thống đã chấm; chưa quá hạn kết thúc giai đoạn 2.

Luồng chính. Student mở kết quả; mở màn hỏi trợ lí, nơi mọi câu sai nằm cùng một màn và phần giải thích không tính giờ; bấm nút làm bài mới để khởi động đồng hồ của lượt; làm câu biến thể của từng câu sai. Câu đúng thì chốt 0,5; câu còn sai thì sang vòng tiếp, tối đa ba vòng, hết ba vòng thì chốt 0.

Ngoại lệ. Hết hạn giai đoạn 2 giữa một lượt thì lượt bị *dừng* và các câu còn dở nhận 0 điểm. Student không mở giai đoạn 2 lần nào thì mọi câu sai nhận 0 điểm.

Quan hệ. *Chữa bài* `<<include>>` *Chấm bài và tra cứu lỗi sai*, và `<<include>>` *Làm câu biến thể, tối đa ba vòng*. *Yêu cầu giải thích cách làm* và *Báo cáo giải thích chưa rõ* đều `<<extend>>` từ nó.

Quan hệ với *Làm câu biến thể* là `<<include>>` chứ không phải `<<extend>>`, **vì giai đoạn 2 bắt buộc**: nó không phải một nhánh có thể không xảy ra. Đây là ví dụ rõ nhất trong cả sơ đồ về việc một quyết định sư phạm đổi luôn kiểu quan hệ trên hình.

5.2.5 UC-05 — Tạo câu biến thể

Điều kiện trước. Một câu ở giai đoạn 1 bị làm sai.

Luồng chính. Hệ thống lấy câu Student làm sai, sinh biến thể của *chính câu đó* — giữ nguyên cấu trúc và lỗi cần kiểm, thay dữ kiện — rồi đưa thẳng tới Student. Kết quả của câu biến thể quyết định câu gốc chốt 0,5 hay sang vòng tiếp.

5.2.6 UC-06 — Báo cáo Trợ lí giải thích chưa rõ

Điều kiện trước. Student đang ở giai đoạn 2, và trợ lí đã trả lời ít nhất một lần.

Luồng chính. Student bấm nút báo cáo "Trợ lí giải thích chưa rõ". Hệ thống ghi lại **cả đoạn hội thoại của bài kiểm tra đó** kèm những câu Student làm sai để giáo viên có thể xem được.

Chương 6

Kiến trúc và mô hình dữ liệu

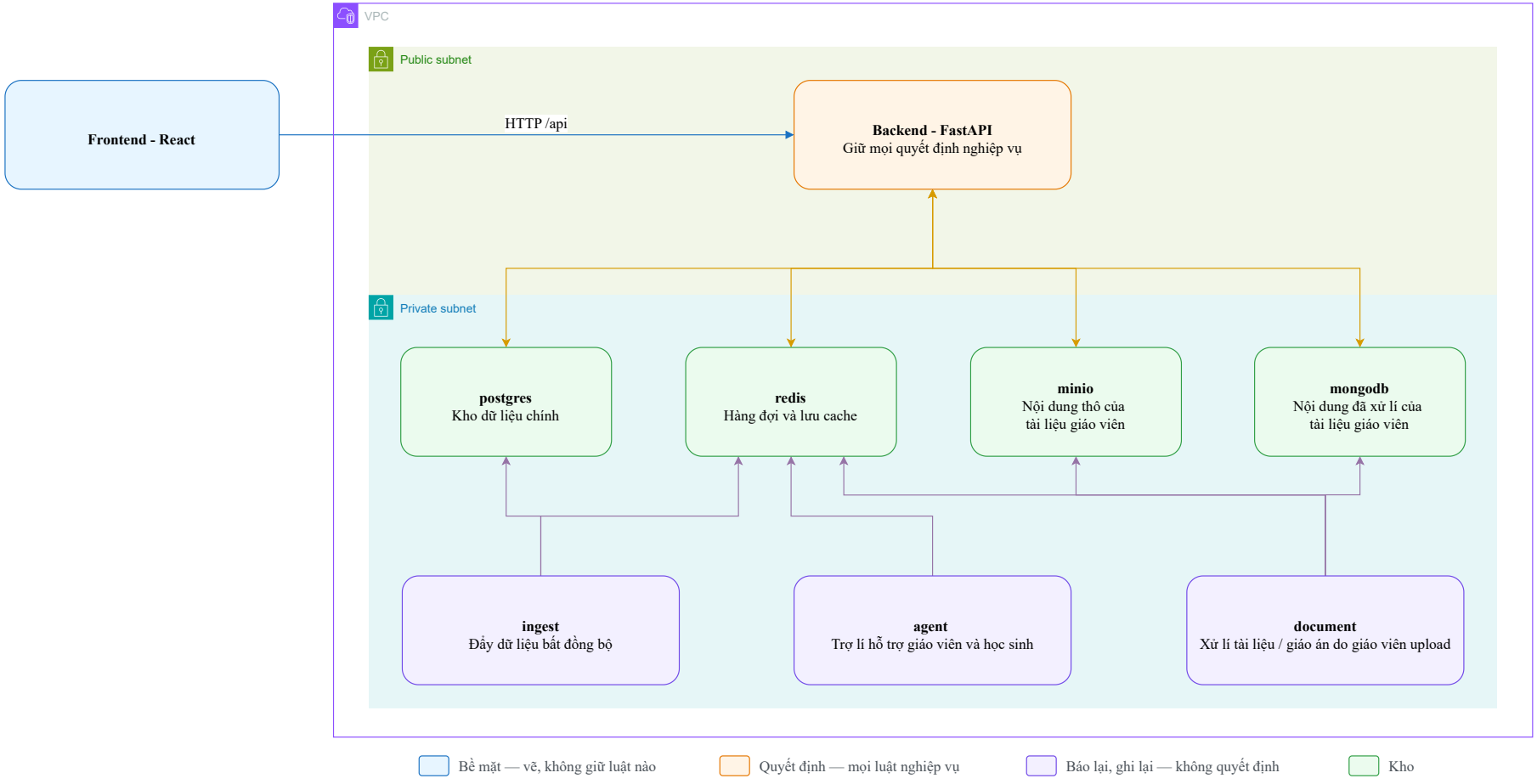
6.1 Các service trong hệ thống

Hệ thống gồm năm **service** (thành phần) chạy độc lập với nhau. Các thuật ngữ tiếng Anh của chương này được định nghĩa ở mục 3.2.

Bảng 6.1: Năm service của hệ thống

Service	Vai trò	Quyết định nghiệp vụ	Credentials
`frontend`	`frontend` cho giáo viên và học sinh. Chỉ nói chuyện với `backend`	Không	Không có
`backend`	Nơi giữ mọi quyết định nghiệp vụ, và là gateway của hệ thống	**Mọi quyết định**	postgres / redis / minio / mongodb
`agent`	Kết nối với Model Provider. Soạn câu hỏi, đề xuất bước tiếp theo cho yêu cầu của giáo viên, và giải đáp học sinh trong hội thoại	Không	redis
`document`	Kết nối với Model Provider. Đọc tài liệu giáo viên tải lên, và dựng cây mục lục cho `agent` dùng khi ra đề	Không	redis / minio / mongodb
`ingest`	Ghi vào database trạng thái xử lí mà `document` báo về	Không	redis / postgres

`frontend` không biết `agent` tồn tại. Mọi thứ nó cần đều đi qua `backend`, nên hệ thống chỉ có một **gateway** (cửa vào duy nhất).



Hình 6.1: System Architecture

6.2 Frontend không nhìn thấy bên trong hệ thống

Hình 6.1 chia hệ thống làm hai vùng, và đặt `frontend` ở ngoài cả hai. Trong **VPC** (đám mây riêng ảo), `backend` nằm ở **public subnet** (vùng mạng công khai); `agent`, `document`, `ingest` và bốn **store** (bốn kho) nằm ở **private subnet** (vùng mạng nội bộ) — chỉ những gì ở public subnet mới nhận được kết nối từ bên ngoài.

`frontend` không bao giờ biết bên trong có mấy service, hay chúng tên gì.

6.3 Backend quyết định mọi thứ

Đây là ràng buộc quan trọng nhất của kiến trúc, và nó là một ràng buộc *nghiệp vụ* chứ không phải một lựa chọn kỹ thuật. Luật chỉ có một câu: **`agent` nộp nội dung / đề xuất hành động, `backend` quyết định làm gì.**

`agent` trả về `backend` hai thứ:

- **Nội dung** — câu hỏi và lời giải cho đề kiểm tra; câu trả lời trong hội thoại với giáo viên hay học sinh; một bản phân tích đọc từ dữ liệu mà `backend` cấp cho nó.
- **Đề xuất hành động** — **tool** (công cụ) nào nên chạy, với tham số nào.

Hai thứ trên có hiệu lực hay không đều do `backend` quyết định. Với nội dung: kiểm trước khi ghi. Với hành động: chỉ chạy những tool chính nó cấp cho mô hình, và từ chối một đề xuất gọi tool ngoài danh sách ấy hay thiếu tham số.

Ví dụ, đối với việc soạn đề:

- `agent` trả về một **đề xuất**: câu hỏi, lời giải, và các phương án lựa chọn, trong đó mỗi phương án nhiễu đi kèm một lỗi cụ thể.
- `backend` kiểm tra lại ràng buộc từng câu rồi mới lưu. Và một đề chỉ rời trạng thái **draft** (nháp) khi giáo viên bấm **approve** (duyệt).

6.4 Service nào kết nối tới store nào?

Bốn store ấy là bốn loại khác nhau: postgres là một **database** (cơ sở dữ liệu) giữ dữ liệu có cấu trúc, redis là một **queue** (hàng đợi), minio là một **object store** (kho tệp) giữ **byte** (dữ liệu gốc) của tệp, và mongodb là một **content store** (kho nội dung) giữ nội dung đã xử lí.

Database thuộc về `backend` và `ingest`. Nó giữ lớp, học sinh, đề, lần làm bài, sổ điểm ba mức, bộ đếm vòng, từng lượt làm lại kèm đề đã sinh ra, các đoạn hội thoại, và các báo cáo.

Object store giữ byte của tài liệu, tức chính tệp giáo viên tải lên. `backend` ghi vào, `document` đọc ra, và không có chiều ngược lại.

Content store giữ thứ đọc được từ tài liệu. Sau khi đọc một tệp, `document` ghi vào nội dung của từng trang và cây mục lục dựng từ chúng. `backend` chỉ đọc ra, để đưa mục lục hay nội dung của một bài cho `agent` khi ra đề theo tài liệu. Tài liệu đã xử lí xong hay chưa thì vẫn do database trả lời: content store giữ *thứ đọc được từ tệp*, không giữ *sự thật về tệp*.

`agent` và `document` không có **credential** (khoá truy cập) của database.

6.5 Các service nói chuyện với nhau thế nào

Hệ thống sử dụng ba hình thức giao tiếp: gọi API trực tiếp, gửi công việc qua hàng đợi và thông báo thay đổi qua Redis Pub/Sub.

Giao tiếp trực tiếp qua API `frontend` gọi `backend`. Đó là đường duy nhất đi vào hệ thống từ bên ngoài, và cũng là lời gọi **API** (giao diện lập trình) duy nhất giữa hai service.

Xử lý công việc qua hàng đợi. Bên gửi đặt một **job** (một đầu việc, một lần chạy) lên queue ở `redis`, kèm **task** là tên của loại việc ấy. Một **worker** (tiến trình nền) ngồi nghe queue đó, lấy job xuống và làm. Bên gửi không bao giờ nói chuyện thẳng với worker; kết quả, nếu cần, cũng đi về qua `redis`.

Thông báo thay đổi qua Pub/Sub. Redis Pub/Sub được sử dụng để thông báo cho các thành phần khác khi có dữ liệu mới hoặc trạng thái thay đổi. Khác với hàng đợi job, thông báo Pub/Sub không được lưu lại để xử lý sau mà chỉ được gửi đến những thành phần đang đăng ký theo dõi channel tại thời điểm phát thông báo.

Ví dụ, khi giáo viên tải lên một tài liệu, `frontend` hiển thị trạng thái "Đang xử lý…", còn `backend` đưa job xử lý tài liệu vào hàng đợi. `document` đọc dữ liệu tệp từ `object store`, trích xuất nội dung và lưu vào `content store`. Sau đó, `document` tạo một job cho `ingest` để cập nhật trạng thái tài liệu trong `database`.

Khi `ingest` hoàn tất việc ghi dữ liệu, nó phát một thông báo lên channel của `redis`. `backend` nhận thông báo này và gửi một event đến `frontend`. Vì event chỉ đóng vai trò báo hiệu, không chứa dữ liệu cập nhật, `frontend` cần gọi lại API để lấy danh sách tài liệu mới nhất và hiển thị trạng thái đã sẵn sàng.

6.6 Job nào đi qua queue, job nào không

Mọi job cần gọi **Model Provider** (nhà cung cấp mô hình) đều chạy qua một queue, đường duy nhất tới `agent` và `document`. Có bảy task như vậy: sáu của `agent`, một của `document`.

Bảng 6.2: Bảy task gọi Model Provider

Task	Service	Khi nào xảy ra
Viết một câu hỏi	`agent`	Mỗi câu của bộ đề giáo viên yêu cầu. **Một câu một job**.
Sinh câu biến thể	`agent`	Khi học sinh nộp bài giai đoạn 1
Trả lời một tin nhắn của học sinh	`agent`	Khi học sinh cần giải đáp thắc mắc ở giai đoạn 2
Đề xuất bước tiếp theo	`agent`	Khi giáo viên gửi một tin nhắn / yêu cầu / thắc mắc
Báo cáo kết quả một plan	`agent`	Sau khi `backend` chạy xong các việc của một plan
Đặt tên một đoạn hội thoại	`agent`	Khi giáo viên gửi tin nhắn đầu tiên và Kriky trả lời
Dựng cây mục lục	`document`	Khi tải lên tài liệu và đã đọc xong nội dung của mọi trang

Chấm bài không đi qua queue. Với trắc nghiệm, chấm là một phép so giữa phương án đã chọn và đáp án đúng đã lưu sẵn, nên nó chạy ngay trong lúc học sinh nộp bài. Kết quả nhìn thấy được trên màn hình: giữa lúc nộp và lúc xem kết quả **không có trạng thái đang chấm nào**.

Màn hình nhận event qua SSE, thay vì phải polling. **SSE** là một kết nối `backend` giữ mở để gửi event xuống khi có gì đổi; **polling** (màn hình tự hỏi lại theo chu kì) là phương án hệ thống không dùng. Có những chỗ dữ liệu hiện ra dần:

- **Hội thoại của học sinh** Hiển thị nội dung phản hồi của trợ lí theo thời gian thực.
- **Hội thoại của giáo viên** Hiển thị nội dung phản hồi của trợ lí, kế hoạch thực hiện (plan) và tiến độ của từng công việc trong kế hoạch theo thời gian thực.
- **Thư viện tài liệu của giáo viên** hiển thị "Đang xử lý…" ngay khi tệp vừa tải lên, rồi tự chuyển sang sẵn sàng mà không cần làm mới màn hình.

6.7 Nếu một service ngừng thì mất gì?

Một kiến trúc chia nhiều phần chỉ đáng giá nếu một phần hỏng không kéo cả hệ thống theo. Bảng 6.3 nói rõ từng trường hợp.

Bảng 6.3: Mỗi service ngừng thì mất gì, còn gì

Ngừng	Mất	Còn
`redis`	Mọi job cần gọi mô hình và tải lên tài liệu	Phát hành đề, làm bài, nộp bài, xem score giai đoạn 1
`minio`	`backend` không khởi động	—
`postgres`	`backend` không khởi động	—
`agent`	Soạn đề, hội thoại	Mọi thứ còn lại
`mongodb`	Ra đề theo tài liệu, và tài liệu mới không xử lí xong	`backend` vẫn khởi động; ra đề không có tài liệu và mọi thứ còn lại
`document`	Tài liệu mới không xử lí xong	Thư viện vẫn mở, tài liệu cũ vẫn dùng được để ra đề
`ingest`	Tài liệu mới không xử lí xong	Job **không mất**: nó nằm lại trên queue, bật lại thì chạy tiếp

6.8 Mô hình dữ liệu

Mô hình dữ liệu có hai mươi hai bảng, chia theo năm nhóm. Mỗi bảng trả lời đúng một câu hỏi nghiệp vụ, và phần ghi chú dưới đây nói vì sao ranh giới giữa chúng nằm ở chỗ đó.

Bảng 6.4: Hai mươi hai bảng, theo năm nhóm

Bảng	Giữ gì
Nhóm 1 — người và lớp	
`classes`	Một lớp giáo viên tạo. Lớp thuộc về giáo viên, không thuộc về nhà trường. Tên lớp *không* phải định danh
`students`	Họ tên, mã học sinh, thuộc lớp nào. Mã học sinh là khoá con người dùng
`teachers`	Họ tên và mã giáo viên
Nhóm 2 — nội dung đề	
`assessments`	Tác giả, tiêu đề, môn, khối, và *trạng thái* — bốn giá trị của vòng đời một đề: rỗng, có câu hỏi, đã duyệt, đã phát hành
`questions`	Đề bài, thứ tự câu học sinh nhìn thấy, mục tiêu học tập
`options`	Nhãn, nội dung, dấu đáp án đúng, và *lỗi mà phương án này đại diện*. Ô lỗi để trống ở đúng một dòng mỗi câu, và dòng đó chính là đáp án đúng
`methods`	Các cách giải của một câu. Bắt buộc nhiều hơn một

tiếp trang sau

Bảng	Giữ gì
`bank_questions`	Ngân hàng câu hỏi của từng giáo viên: bản chép của mỗi câu, kèm phương án và lời giải, lúc đề chứa nó được duyệt. Xoá câu khỏi đề thì bản này ở lại
`publications`	Sáu tham số phát hành, một dòng cho mỗi cặp đề và lớp, nên thu hồi được từng lớp một
`documents`	Tài liệu giáo viên tải lên: tên tệp, kích thước, khoá trỏ tới object store, trạng thái xử lý, số trang, số trang đã đọc, lý do hỏng. **Không giữ byte nào**. Thuộc về giáo viên, không thuộc về đề
`draft_briefs`	Yêu cầu soạn của một đề: môn, khối, phạm vi, tổng số câu đã xin; tài liệu và các nút của cây mục lục khi ra đề theo tài liệu; và các mục dàn ý giáo viên đã đồng ý
`draft_items`	Một ô cho mỗi câu đang soạn, kèm mục dàn ý câu ấy theo, trạng thái, số lần đã thử, và *lý do* lần gần nhất bị loại
Nhóm 3 — bài làm và điểm	
`attempts`	Một học sinh làm một đề: lớp lúc bắt đầu, giờ bắt đầu, giờ phải xong, giờ đã nộp
`answers`	Lựa chọn ở giai đoạn 1, ghi mỗi lần bấm chứ không gom tới lúc nộp
`question_outcomes`	**Sổ điểm**: mark cuối, lý do, số vòng đã dùng, đã chốt hay chưa
Nhóm 4 — giai đoạn 2	
`rounds`	Một lượt làm lại và giờ kết thúc của nó, đã cắt xuống hạn giai đoạn 2 nếu vượt
`round_items`	Câu sinh cho lượt đó, kèm phương án và lời giải — giữ cả đáp án đúng ở đây là thứ cho phép hệ thống tự chấm lượt
`pregenerated_items`	Câu viết *trước* khi học sinh bấm nút, trong lúc em còn đang đọc giải thích
`chat_messages`	Từng tin nhắn giữa học sinh và trợ lí. Câu trả lời hiện dần lên màn hình và chỉ được lưu khi đã trọn vẹn; đóng thẻ trình duyệt giữa chừng thì câu ấy mất, và trợ lí chỉ trả lời lại khi em gửi tin nhắn tiếp theo
`reports`	Báo cáo *giải thích chưa rõ*, gắn với cả bài chứ không với một tin nhắn
Nhóm 5 — hội thoại của giáo viên	

tiếp trang sau

Bảng	Giữ gì
`teacher_conversations`	Một đoạn hội thoại của một giáo viên. Tiêu đề do trợ lí đặt sau lượt hội thoại đầu tiên, giáo viên sửa được; xoá là *xoá mềm*
`teacher_turns`	Từng *bước* của một lượt hội thoại: lời nói, plan, dàn ý, hoặc tool đã chạy, kèm chủ thể mà bước đó nói về

Thứ đọc được từ tài liệu không nằm trong hai mươi hai bảng này mà ở content store; database chỉ biết một tài liệu đã xử lí xong hay chưa.

Chương 7

Chi tiết các service

Chương 6 nhìn hệ thống từ bên ngoài: có những **service** (thành phần) nào, mỗi service giữ khoá của **store** (kho) nào, và chúng nói chuyện với nhau ra sao. Chương này đi vào bên trong ba service gánh phần lớn việc xử lí của hệ thống: `backend`, `agent` và `document`. Mỗi mục trả lời năm câu hỏi theo cùng một thứ tự: service nhận gì và trả gì, bên trong gồm những phần nào, một luồng đi qua nó ra sao, nó giữ những luật nào và cái gì canh các luật ấy, và khi có gì hỏng thì nó làm gì. Ba tiêu đề *Vào và ra*, *Luật và chỗ canh luật* và *Khi hỏng* giống nhau ở cả ba mục, để đặt chúng cạnh nhau mà so. Bốn service viết bằng Python không dùng mã nguồn của nhau, và `lint-imports` canh điều ấy cho cả bốn, nên ba bảng luật dưới đây không nhắc lại.

7.1 `backend`

`backend` là service duy nhất `frontend` gọi tới. Bên trong, nó chỉ có một tiến trình: tiến trình ấy trả lời lời gọi, giao việc đi qua **queue** (hàng đợi), rồi lấy kết quả về theo ba cách.

7.1.1 Vào và ra

Từ `frontend`, `backend` nhận ba mươi sáu loại lời gọi **API** (giao diện lập trình): mười sáu cho học sinh, mười chín cho giáo viên, và một để kiểm tra tiến trình còn sống. Ba trong số đó trả về bằng **SSE** (kết nối giữ mở), đúng ba chỗ chương 6 đã kể: qua kết nối ấy, `backend` gửi từng **event** (sự kiện) xuống màn hình.

Đi vào trong hệ thống, `backend` giao **job** (một đầu việc, một lần chạy) qua queue của `redis`. Các job ấy thuộc sáu **task**, tức sáu loại việc, của `agent` trong bảng 6.2, cùng một task dò tệp cho `document`; task dò tệp không gọi mô hình nên không có trong bảng ấy. `backend` cũng nghe ba loại **channel** (kênh). Hai loại chỉ mang thông báo: khi một câu của đề đang soạn vừa xong, và khi tài liệu của một giáo viên vừa đổi trạng thái hay đọc thêm được trang. Loại thứ ba mang chữ hiện dần của từng câu trả lời cho học sinh.

7.1.2 Bảy mảng việc, và không có worker nào

Mã nguồn của `backend` chia thành bảy mảng việc (bảng 7.1). Ranh giới mảng không trùng ranh giới tệp: một luồng như soạn đề đi qua ba mảng, và một tệp có thể thuộc nhiều mảng.

backend không có **worker** (tiến trình nền) nào: tiến trình duy nhất của nó chỉ chạy khi có lời gọi. Vì vậy kết quả của những job nó giao đi mà không đứng đợi chỉ được lấy về khi có một lời gọi khác đi qua — việc mà báo cáo gọi là *thu hoạch* (mục 3.2). Thu hoạch chạy cả trong lời gọi chỉ đọc, nên ngay lời gọi xem chi tiết một đề cũng có thể ghi câu mới vào **database** (cơ sở dữ liệu).

Bảng 7.1: Bảy mảng việc bên trong backend

Mảng	Việc
Khung tiến trình	Mở kết nối tới các store lúc khởi động, đọc cấu hình, nạp dữ liệu mẫu khi database còn trống
Vòng đời đề	Duyệt, bỏ duyệt, phát hành, thu hồi; sửa và xoá câu; chép câu vào ngân hàng khi duyệt
Soạn đề từng câu	Mỗi câu một job; thu hoạch kết quả, kiểm từng câu, đặt câu đúng vị trí
Cổng sang agent	Ba cách lấy kết quả một job; kiểm câu hỏi trước khi ghi
Hội thoại giáo viên	Chạy lượt hội thoại hai pha, cấp tool theo pha, kiểm plan, khoá đoạn hội thoại
Học sinh	Làm bài, chấm ngay trong lời gọi nộp, giai đoạn 2, hội thoại với trợ lí
Tài liệu	Cất byte của tệp trước rồi mới ghi hàng; đọc cây mục lục và chunk; nghe thông báo

7.1.3 Ba cách lấy kết quả một job

Queue tách backend khỏi agent, nhưng không tách màn hình khỏi backend: khi có người đang nhìn màn hình, backend giữ lời gọi của màn hình mở trong lúc lấy kết quả job. Có ba cách lấy kết quả, chọn theo việc ai cần kết quả ấy và cần lúc nào (bảng 7.2).

Bảng 7.2: Ba cách backend lấy kết quả một job, và giới hạn mặc định của mỗi cách

Cách	Dùng cho	Giới hạn
Lấy ngay trong lời gọi	Ba task của hội thoại giáo viên; câu biến thể khi phải viết tại chỗ	70 giây cho một job
Nghe trước, rồi mới giao job	Trả lời một tin nhắn của học sinh	25 giây không có chữ mới
Giao job rồi đi, thu hoạch sau	Viết một câu hỏi; viết trước câu biến thể	Kết quả sống 1 giờ

Cách thứ hai nghe trước rồi mới giao job, vì **pub/sub** (phát lên một channel, ai đang nghe thì nhận) của redis không giữ lịch sử: giao job trước thì những chữ đầu tiên agent viết ra có thể rơi vào lúc chưa ai nghe. Channel soạn đề cũng được nghe trước khi giao job, theo đúng lý do ấy.

7.1.4 Các luồng đi qua backend

Năm đoạn dưới đây đi theo thứ tự một đề ra đời rồi tới tay học sinh. Mỗi đoạn chỉ nói điều backend làm mà chương 4 không nói: cái gì được kiểm, và cái gì bị khoá.

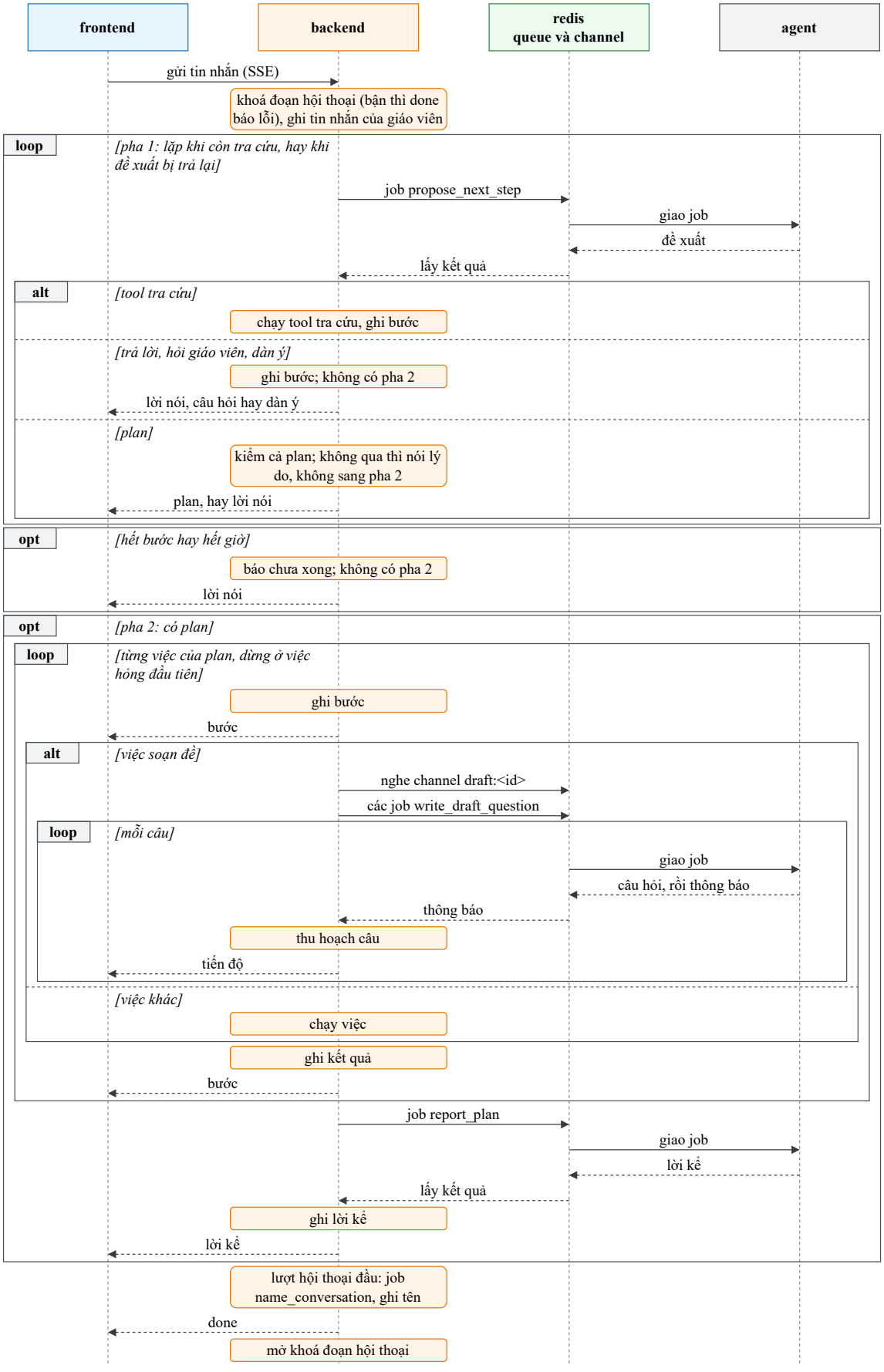
Lượt hội thoại của giáo viên. Hình 7.1 vẽ trọn một lượt hội thoại. Trước khi ghi gì, backend khoá đoạn hội thoại bằng một câu lệnh duy nhất trên database, nên khi hai thẻ trình duyệt cùng gửi thì chỉ một thẻ thắng; thẻ kia nhận một event lỗi ngay trong luồng SSE. Ở pha 1, agent chỉ *gọi* được **tool** (công cụ) tra cứu: tool ghi chỉ được nêu trong một **plan** (danh sách việc trợ lí định làm), và một lời gọi tool ghi ở pha 1 bị từ chối như gọi một tool không tồn tại. Pha 1 lặp chừng nào agent còn tra cứu, hay đề xuất của nó bị trả lại, trong trần mặc định tám bước và chín mươi giây; mỗi phần tài liệu được đọc thêm cộng một bước và mười giây, tối đa mười phần. Khi agent nêu một plan, backend kiểm *cả* plan trước khi việc nào chạy: không quá tám việc, mỗi việc gọi một tool có trong danh sách của pha 2, đủ tham số, và một việc dùng kết quả của việc khác thì chỉ được trỏ tới việc đứng trước nó. Một plan soạn câu còn phải theo đúng dàn ý giáo viên vừa xem ở lượt hội thoại ngay trước. Pha 2 chạy từng việc và dừng ở việc hỏng đầu tiên, rồi giao một job riêng để agent kể lại kết quả. Đoạn hội thoại được mở khoá ở cuối, kể cả khi lượt hội thoại hỏng giữa chừng.

Soạn đề từng câu. Một yêu cầu soạn mười câu thành mười job, mỗi job một vị trí. Đề còn câu đang soạn thì không mở được lần soạn mới, nên mọi câu của một lần soạn theo cùng một yêu cầu. Thu hoạch được một câu, backend cho nó qua đúng phép kiểm mà chương 6 nêu, cộng một phép chống trùng đề bài với các câu đã có, rồi đặt nó vào đúng vị trí giáo viên xin, không theo thứ tự nó về. Câu bị loại thì vị trí ấy được giao lại, trong giới hạn ba lần thử. Thông báo mỗi câu soạn xong chỉ đánh thức backend đi thu hoạch; con số trên màn hình luôn được đếm lại từ database, vì một thông báo có thể mất.

Duyệt, bỏ duyệt, phát hành. Duyệt chỉ đi qua khi backend đã thu hoạch mọi câu vừa soạn xong, không còn câu nào đang được soạn, và đề có ít nhất một câu; trong cùng một lần ghi ấy, mọi câu được chép vào ngân hàng câu hỏi. Từ lúc đó, mọi đường ghi câu bị từ chối cho tới khi giáo viên bỏ duyệt. Một lần phát hành dùng chung một khung giờ cho mọi lớp được chọn nhưng xét từng lớp riêng: phát hành lại cho một lớp đã qua giờ mở, hay đã có học sinh vào làm, thì chỉ lớp ấy trượt. Thu hồi chỉ được trước giờ mở của lớp đó, và bỏ duyệt một đề đã phát hành là thu hồi mọi lớp trong một lần ghi: hoặc tất cả, hoặc không lớp nào.

Làm bài và chấm. Giờ phải xong của một **Attempt** (lần làm bài) do backend đặt lúc học sinh bắt đầu; đồng hồ trên màn hình chỉ để hiển thị, còn một lựa chọn gửi tới sau giờ ấy bị từ chối. Đáp án đúng không tới màn hình học sinh trước khi em nộp: câu gửi xuống không mang dấu đáp án nào, và lời gọi xem lời giải bị từ chối khi chưa nộp. Nộp xong, backend giao job viết trước câu biến thể cho những câu sai.

Giai đoạn 2. Thời lượng của một lượt bị cắt xuống hạn kết thúc giai đoạn 2 nếu vượt. Học sinh mở một lượt mà câu biến thể của mọi câu còn mở đã được viết sẵn thì lượt mở ngay; chưa có thì backend giao một job mới và lấy kết quả ngay trong lời gọi mở lượt. Sau mỗi lượt, điểm của một câu chỉ có thể lên, không xuống.



Hình 7.1: Một lượt hội thoại của giáo viên, qua SSE

7.1.5 Luật và chỗ canh luật

Bảng 7.3 ghi những luật backend giữ, nơi thi hành chúng, và loại lưới canh chúng.

Bảng 7.3: Luật backend giữ, nơi thi hành, và loại lưới canh chúng

Luật	Nơi thi hành	Loại lưới
Không tool nào đổi trạng thái của đề	danh sách tool trong teacher_tools	check_contract
Đề đã duyệt từ chối mọi đường ghi câu	assessment_state	kiểm thử tự động
Câu hỏi được kiểm trước khi ghi	validate_question trong agent_gateway	kiểm thử tự động
Không câu nào soạn từ một dàn ý giáo viên chưa thấy; job soạn câu không nhận chữ thô của tài liệu	guard_drafting trong teacher_chat; drafting	check_contract, kiểm thử tự động
Đề, câu hay lớp của giáo viên khác trả lời như không tồn tại	teacher_routes	kiểm thử tự động
Một đoạn hội thoại chạy nhiều nhất một lượt hội thoại một lúc	teacher_chat	kiểm thử tự động
Một đề rỗng không duyệt được; một đề chưa duyệt không phát hành được	teacher_routes	kiểm thử tự động
Mỗi học sinh có nhiều nhất một Attempt cho một đề	bảng attempts	ràng buộc database
Byte của tệp được cất trước, hàng tài liệu ghi sau	teacher_documents	kiểm thử tự động

Ba luật chưa được ép, và mỗi luật có một chỗ hở cụ thể. Luật giờ kết thúc của một lượt (chương 4): lưu từng lựa chọn thì có kiểm giờ, nhưng nộp cả lượt thì không, nên một lượt nộp muộn vẫn được nhận. Luật câu biến thể giữ cấu trúc của câu gốc (mục 3.2): ngoài các phép kiểm chung của mọi câu, chỉ có phép so đề bài với câu gốc và với các vòng trước; không gì so cấu trúc. Luật một plan đã chạy thì chạy hết và được kể lại: giáo viên đóng thẻ trình duyệt giữa pha 2 thì lượt hội thoại dừng ngay chỗ đang đợi, các việc còn lại không chạy, và không có lời kể nào.

7.1.6 Khi hỏng

Bảng 6.3 nói một service ngừng thì hệ thống mất gì. Bảng 7.4 nói backend làm gì khi một thao tác bên trong nó hỏng.

7.2 agent

agent là service soạn nội dung bằng mô hình ngôn ngữ. Nó không mở cổng nào: nó là một worker ngồi nghe queue của mình, gọi **Model Provider** (nhà cung cấp mô hình), rồi trả kết quả về redis.

Bảng 7.4: Khi một thao tác bên trong backend hỏng

Khi	backend làm gì
Không lấy được kết quả từ agent trong lúc lời gọi còn mở	Lượt hội thoại kết thúc bằng một event lỗi; lời kể lại thay bằng một câu backend tự dựng từ kết quả các việc; tên đoạn hội thoại lấy câu đầu tiên đã làm gọn; mở lượt làm lại trả lỗi 503
Quá lâu không có chữ mới khi đang trả lời học sinh	Báo lỗi ngay trong luồng SSE; câu trả lời dở không được lưu
Kết quả một job hết hạn trước khi được thu hoạch	Giao lại job; với việc soạn đề, lần ấy tính vào ba lần thử
Câu không qua phép kiểm, hay trùng câu đã có	Giao lại, trong giới hạn ba lần thử, rồi dừng kèm lý do
Job soạn câu hỏng hẳn	Vị trí ấy dừng ngay, không giao lại
Tiến trình chết giữa một lượt hội thoại	Khoá của đoạn hội thoại có hạn, mặc định 10 phút, nên đoạn không kẹt mãi
Không giao được job dò tài liệu	Hàng tài liệu ghi ngay là hỏng, kèm lý do
Tài liệu không báo tiến độ quá 300 giây	Đọc ra là hỏng, mà không ghi gì

7.2.1 Vào và ra

agent nhận job của sáu task, và mỗi task chạy trong đúng một **graph** (chuỗi khâu cố định, dựng bằng thư viện LangGraph); hai task soạn câu dùng chung một graph. Bảng 7.5 nói mỗi task đi qua graph nào, gọi mô hình mấy lần và trả về gì; còn khi nào mỗi task được giao thì bảng 6.2 đã nói.

Mỗi job tự chứa mọi thứ nó cần: job trả lời học sinh mang theo cả câu hỏi, các phương án, lời giải, đáp án đúng và phương án học sinh đã chọn, chứ không mang một mã câu hỏi rồi tự đi tra. Ngoài giá trị trả về, agent còn phát hai thứ qua pub/sub: từng mảnh chữ của câu trả lời cho học sinh, và thông báo mỗi khi viết xong một câu cho đề đang soạn.

7.2.2 Bốn phần bên trong

Vỏ worker. Sáu task được đăng ký dưới đúng tên khai trong gói dùng chung contracts, không dưới tên hàm, nên đổi tên một hàm không làm gãy lời giao job của backend. Thông báo của write_draft_question chỉ được phát sau khi kết quả đã ghi vào redis, không phát trong thân job: phát sớm thì backend nhận thông báo, đi thu hoạch, và chỉ gặp một kết quả chưa có.

Hàm nhận job, và ba lối ra. Mỗi task có một hàm nhận job, và cả sáu hàm đi cùng một đường. Mô hình bị tắt trong cấu hình thì hàm trả *nội dung dọn trước*, tức câu chữ viết sẵn trong mã nguồn; mô hình bật thì hàm chạy graph; graph hỏng thì hàm quay về nội dung dọn trước. Riêng hàm trả lời học sinh: đã phát được vài chữ rồi mới hỏng thì nó giữ đúng những chữ ấy, vì học sinh đã đọc chúng.

Bảng 7.5: Sáu task của `agent`: graph, số lần gọi mô hình, và thứ trả về

Task	Graph	Gọi mô hình	Trả về
`write_draft_question`	`authoring`	Một tới ba lần	Một câu hỏi
`generate_retry_question`	`authoring`	Một tới ba lần	Một câu biến thể
`explain_turn`	`explain`	Một lần, trả chữ dần	Lời giải thích
`propose_next_step`	`propose`	Một hay hai lần, sau khi Jev chọn lời dặn	Một đề xuất: trả lời, hỏi giáo viên, dàn ý, tool, hay plan
`report_plan`	`reporting`	Một lần	Lời kể lại
`name_conversation`	`naming`	Một lần	Tên đoạn hội thoại

Graph và khâu. Mọi lần `agent` gọi mô hình để viết chữ đều nằm trong một graph. Graph soạn câu có hai khâu, viết rồi tự kiểm: câu không qua phép tự kiểm thì quay lại khâu viết kèm lý do bị loại, và khâu viết chạy tối đa ba lần. Phép tự kiểm này không thay cho phép kiểm của `backend`: nó chỉ để khỏi mất một vòng gửi đi rồi bị từ chối.

Nối mô hình. Chỉ một tệp, `llm.py`, nối tới mô hình ngôn ngữ; Jev, mô hình phân loại của TypeSafe AI, đi qua một tệp riêng. `llm.py` truyền khoá vào, đặt trần mặc định hai mươi giây cho mỗi lần gửi, và xếp một mô hình dự phòng sau mô hình chính nếu cấu hình có. Mô hình trả lời theo một khuôn JSON cố định, và chữ trong đó — câu hỏi soạn ra, lời nói của một đề xuất, tên các việc của plan, dàn ý — còn đi qua một bộ dựng lại ký hiệu toán: JSON nuốt dấu gạch chéo của LaTeX, nên `\frac` về tới nơi thành một ký tự điều khiển cộng với "rac". Lỗi này đã lọt vào database thật ngày 05/10/2026.

7.2.3 Một bước của lượt hội thoại: ba tầng phòng thủ

Đề xuất bước tiếp theo là task phức tạp nhất của `agent`: graph `propose` chiếm hơn một phần ba mã nguồn và khoảng một nửa số ca kiểm thử của service. Mỗi bước của pha 1 trong hình 7.1 đi qua ba tầng, và mỗi tầng bắt một loại sai khác.

Tầng một: mô hình đề xuất trong một khuôn. Lời dặn cho mô hình được ghép từ một lõi chung, vài tệp lời dặn theo chủ đề, tên giáo viên, tài liệu đang đính kèm, và danh sách tool. Tệp lời dặn nào được nạp thì Jev chọn: nó đọc bốn lượt chữ gần nhất rồi trả lời có hay không cho từng tệp chuyên biệt; không tệp nào được chọn thì nạp tệp chung, còn Jev không trả lời thì nạp cả ba. Loại đề xuất trong khuôn là một trong năm: trả lời, hỏi giáo viên, đưa ra dàn ý, gọi một tool, hay nêu plan. Khuôn chỉ chứa những gì mô hình trả lời được, vì mỗi ô trong khuôn là một câu hỏi đặt cho nó. Bài học ấy có giá: khi khuôn còn là cả bản hợp đồng giữa hai service, nó có một ô mở tới mức nhà cung cấp mô hình từ chối thẳng mọi lần gọi, và nội dung dọn trước che kín chuyện ấy sau một câu trả lời nghe hợp lý.

Tầng hai: luật trong `agent` sửa đề xuất. Một số luật sửa đề xuất ngay trong `agent`, trước khi nó rời đi. Ví dụ: giáo viên vừa được xem dàn ý và trả lời bằng một câu chỉ gồm lời đồng ý, như "ok" hay "được", thì `agent` dựng plan thẳng từ dàn ý, không giữ plan mô hình tự viết,

vì sau một chữ "ok" mô hình hay quên phạm vi vừa bàn. Sau các luật sửa là các phép kiểm; đề xuất vấp một phép kiểm thì `agent` gọi mô hình thêm đúng một lần.

Tầng ba: `backend` gác cổng. Đề xuất về tới `backend` vẫn chỉ là đề xuất, và mục 7.1 đã kể `backend` kiểm gì. Nhưng có một giới hạn nên nói thẳng: `backend` chứng minh được dàn ý đã hiện ra cho giáo viên, không chứng minh được giáo viên đã đồng ý. Lời đồng ý chỉ được đọc trong `agent`: nếu mô hình đọc "ok nhưng đổi câu 2" thành đồng ý và nêu plan soạn câu, `backend` vẫn cho qua.

7.2.4 Luật và chỗ canh luật

Bảng 7.6 ghi những luật `agent` giữ, nơi thi hành chúng, và loại lưới canh chúng.

Bảng 7.6: Luật `agent` giữ, nơi thi hành, và loại lưới canh chúng

Luật	Nơi thi hành	Loại lưới
Không trả về phán quyết nào, ở bất kỳ độ sâu nào của kết quả	mọi hàm nhận job	kiểm thử tự động
Không giữ credential của database nào	cả service	`check_contract`
Một job, kể cả mọi lần thử lại, hết giờ trước khi `backend` thôi lấy kết quả	cấu hình của hai service	`check_contract`
Lời kể lại là một job riêng, không cầm tool nào	`reporting`	`check_contract`
Chữ trả về theo khuôn đi qua bộ dựng lại ký hiệu toán	`latex_escapes`	`check_contract`
Câu hỏi có đúng một đáp án đúng, mọi phương án nhiễu gắn một lỗi, ít nhất hai lời giải	phép tự kiểm trong `authoring`	kiểm thử tự động
Lời kể lại không nói quá điều đã làm	chỉ lời dặn của `reporting`	không gì canh

Hai lưới trong bảng yếu hơn tên của chúng. Ca kiểm thử "không phán quyết" chạy mọi task hai lần, với mô hình tắt rồi bật, nhưng mô hình giả dùng khi kiểm thử không dựng được đầu ra theo khuôn: ở lần chạy bật, ba trong sáu task vẫn rơi về nội dung dọn trước, nên giá trị thật của hai graph `authoring` và `propose` chưa bị soi. Và phép kiểm thời gian chưa tính mô hình dự phòng, cũng chưa tính hai lần thư viện của OpenAI tự gửi lại khi quá giờ, nên một lần gọi có thể dài gần sáu mươi giây.

7.2.5 Khi hỏng

Nội dung dọn trước là một lựa chọn có chủ ý: thà có một câu còn hơn một tính năng từ chối chạy. Nhưng nó có cái giá, và bảng 7.7 ghi cả cái giá ấy.

Bảng 7.7: Khi một thao tác bên trong `agent` hỏng, và cái giá của cách nó đỡ

Khi	`agent` làm gì	Cái giá
Mô hình quá giờ hay báo lỗi	Gọi mô hình dự phòng nếu có, rồi trả nội dung dọn trước	Lỗi bị che sau một câu nghe hợp lý
Câu hay đề xuất vấp phép tự kiểm	Viết lại trong graph: câu hỏi tới tối đa ba lần, đề xuất thêm một lần	Câu hỏi vẫn vấp sau ba lần thì hàm nhận job trả câu dọn trước, như hai hàng dưới; đề xuất vẫn sai thì vẫn tới `backend`, hay thành một câu trung tính
Mô hình hỏng hẳn khi soạn một câu hỏi	Trả một câu dọn trước	Câu ấy lấy từ sáu câu Toán cố định, không theo môn hay phạm vi giáo viên xin, và `backend` không có dấu nào để nhận ra; chỉ cổng duyệt của giáo viên chặn nó
Mô hình hỏng hẳn khi viết câu biến thể	Câu gốc nằm trong bộ câu dọn trước và còn biến thể chưa dùng thì trả biến thể ấy; không thì trả lại chính câu gốc, thêm đuôi "(đề thay số cho lượt …)"	Khi mô hình bật, câu gốc do mô hình viết nên luôn rơi vào nhánh sau: câu ấy qua được phép kiểm của `backend`, và học sinh làm lại đúng câu cũ mà không cổng nào chặn
Mô hình hỏng giữa lúc trả lời học sinh	Giữ những chữ đã phát	Câu trả lời bị cắt ngang
Mô hình hỏng khi đề xuất bước tiếp theo	Trả một đề xuất dọn trước, đoán việc bằng từ khoá	Đề xuất ấy có thể là một plan thật
Jev không trả lời	Nạp cả ba tệp lời dặn	Lời dặn dài hơn, và luật của những tệp không liên quan cũng được áp

7.3 `document`

`document` đọc những tệp giáo viên tải lên và dựng cây mục lục cho chúng. Khi một job của nó bắt đầu, **byte** (dữ liệu gốc) của tệp đã nằm trong **object store** (kho tệp) và job đã nằm trên queue (mục 6.5); mục này kể từ lúc worker lấy job xuống tới lúc kết quả rời service.

7.3.1 Vào và ra

`document` nhận job của ba task: `probe_document` từ `backend`, cùng hai task nó tự giao cho chính mình, `ocr_pages` cho lô **OCR** (đọc chữ từ ảnh) kế tiếp và `index_document` cho việc dựng cây. Nó gửi về queue của `ingest` job của hai task: `document_probed` mang kết quả, `document_progress` báo tiến độ. Job tới mang khoá của tệp trong object store chứ không mang byte của nó. Mọi thứ `document` ghi đều nằm ở **content store** (kho nội dung), chia thành sáu **collection** (nhóm bản ghi cùng loại, như một bảng) mà bảng 7.8 liệt kê.

Bảng 7.8: Sáu collection của content store: giữ gì, ai ghi, ai đọc

Collection	Giữ gì	Ai ghi	Ai đọc
`documents`	Đầu mục của bản thô, khác bảng cùng tên của database: đọc thẳng chữ có sẵn hay qua OCR, đã đọc xong chưa	đường đọc tệp	đường dựng cây
`pages`	Chữ của từng trang, chia theo khối	đường đọc tệp	đường dựng cây
`indexes`	Đầu mục của cây, ghi sau cùng	đường dựng cây	`backend`
`nodes`	Các nút của cây mục lục	đường dựng cây	`backend`
`chunks`	Các chunk của mỗi nút	đường dựng cây	`backend`
`block_labels`	Quyết định cho từng ứng viên tiêu đề: ai quyết, Jev đã phân vân giữa những gì; để gỡ lỗi và đo	đường dựng cây	không ai, lúc chạy

7.3.2 Vì sao có hai worker

Đọc tệp và dựng cây bị giới hạn bởi hai thứ khác nhau. Worker đọc tệp chạy một job một lúc: OCR chiếm trọn bộ xử lí, nên hai lô chạy song song chỉ tranh nhau rồi cùng chậm. Nó nạp ba mô hình OCR một lần lúc khởi động, vì nạp lại ở mỗi lô là trả lại chừng ấy thời gian cho từng lô của một cuốn. Worker dựng cây không nạp mô hình nào và chạy tới bốn job song song, vì phần lớn thời gian của nó là đợi Model Provider trả lời.

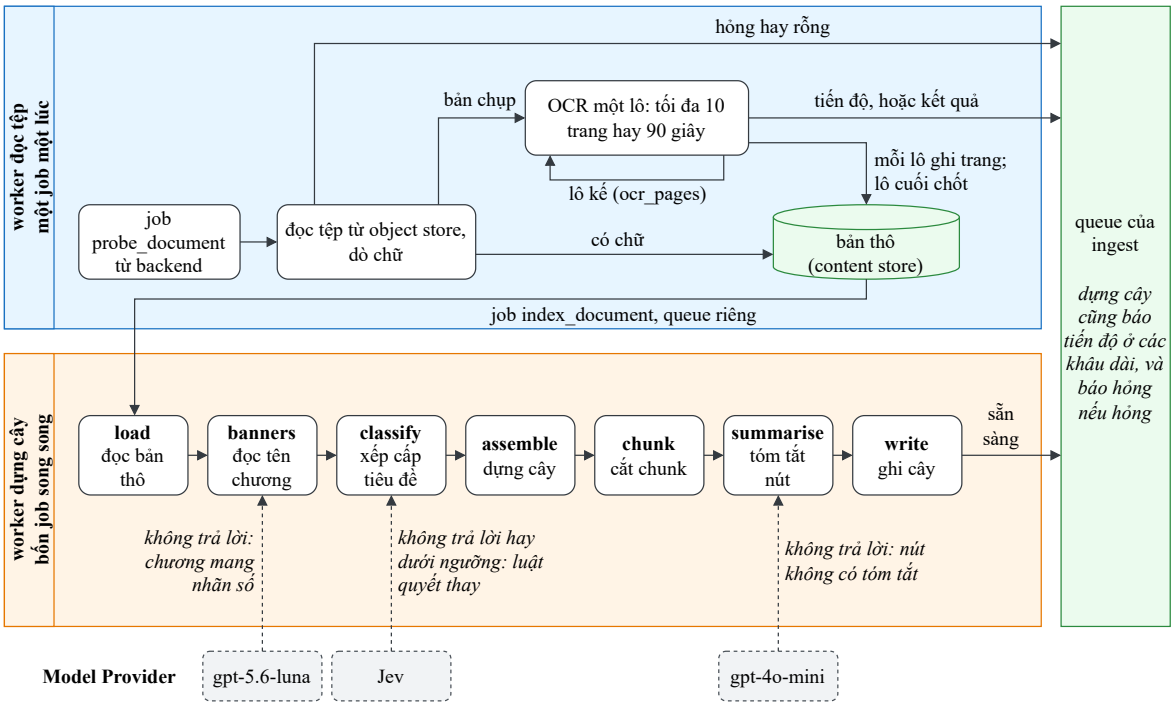
Cái giá của một job một lúc là nhiều bản chụp tải lên cùng lúc được đọc xen kẽ từng lô. Từ chừng bảy cuốn trở lên, hay bốn cuốn nếu trang dày chữ, một cuốn im quá ba trăm giây giữa hai lần báo tiến độ, và `backend` đọc mọi cuốn đang đọc ra là hỏng.

7.3.3 Một tệp đi qua service

Hình 7.2 vẽ đường đi của một tệp qua hai worker.

Dò tệp. Worker đọc tệp lấy byte của tệp từ object store rồi dò xem nó có chữ không. Tệp `.txt` và `.md` phải là UTF-8, hay UTF-16 có dấu BOM; bảng mã khác bị từ chối chứ không đoán, vì đoán sai thì ra chữ trông như chữ mà sai dấu. Với tệp PDF, nó đếm số trang có chữ, và một trang tính là có chữ khi có từ một trăm ký tự trở lên: từ mười phần trăm số trang có chữ trở lên thì tệp có sẵn chữ, còn lại là bản chụp. Một PDF không có trang nào bị trả về là không có chữ, không đi OCR. Đếm trang chứ không cộng ký tự là có chủ ý: một bản chụp có một trang mục lục dày chữ vẫn không lọt cổng.

Bản chụp đọc theo lô. Một bản chụp được đọc qua nhiều job, mỗi job một lô: tối đa mười trang, và không mở trang mới sau chín mươi giây. Mỗi lô ghi trang của nó vào content store; trừ lô cuối, nó báo tiến độ về `ingest` rồi tự giao lô kế lên chính queue của worker đọc tệp. Mỗi trang đi qua năm chặng: dựng ảnh, nhận khối, tìm dòng, đọc chữ từng dòng, rồi gán dòng vào khối. Hai chặng nhận khối và tìm dòng dùng mô hình của PaddleOCR, còn chặng



Hình 7.2: Một tệp đi qua `document`: hai worker, ba mô hình ngoài, và các đường lùi

đọc chữ dùng EasyOCR, vì bộ đọc chữ của PaddleOCR không bao giờ phát ra chữ tiếng Việt mang hai dấu như "ế" hay "ộ". Công thức toán không được đọc riêng, vì lần đo trên máy chỉ có bộ xử lí trung tâm cho thấy mô hình đọc công thức không cứu được: khối công thức chỉ nhận chữ OCR như mọi khối khác.

Bản thô ghi trước. Đọc xong, `document` có một *bản thô* (mục 3.2) trong content store, nhưng tài liệu *chưa* sẵn sàng: nó còn cần cây mục lục. Worker đọc tệp giao một job `index_document` lên queue riêng của worker dựng cây, và chưa gửi kết quả nào về `ingest`.

Bảy khâu dựng cây. Job dựng cây chạy một graph gồm bảy khâu nối thẳng (bảng 7.9). Ba khâu gọi một mô hình ngoài, và cả ba đều có đường lùi ngay bên trong khâu: mô hình không trả lời thì tài liệu vẫn sẵn sàng, chỉ kém hơn.

Bảng 7.9: Bảy khâu dựng cây mục lục, và đường lùi của những khâu gọi mô hình

Khâu	Làm gì	Mô hình không trả lời thì
`load`	Đọc bản thô; bản thô chưa đọc xong thì không chạy	—
`banners`	Đọc tên chương trên ảnh đầu chương, bằng `gpt-5.6-luna`	Chương mang nhãn số
`classify`	Xếp vai trò và cấp cho từng tiêu đề; luật và Jev cùng quyết, Jev thắng khi đủ chắc	Luật quyết thay
`assemble`	Dựng cây tối đa ba cấp, không nhảy cấp; tiêu đề sâu hơn gộp vào nút cha	—
`chunk`	Cắt chữ thành các chunk dưới từng nút	—
`summarise`	Tóm tắt từng nút, từ dưới lên, bằng `gpt-4o-mini`	Nút không có tóm tắt
`write`	Ghi cây vào content store	—

Mô hình không dựng cây. Ở khâu `classify`, luật và Jev cùng đưa ra một quyết định cho mỗi tiêu đề, và Jev chỉ thắng khi độ tin cậy của nó đạt một ngưỡng cấu hình được. Trên cuốn duy nhất đã có mục lục chuẩn để so, ở ngưỡng hiện tại Jev chưa đổi quyết định nào của luật, nên cây của cuốn ấy chính là cây của luật. Luật cũng được chỉnh trên chính cuốn ấy, nên phép so này chưa công bằng.

Ghi cây rồi mới báo sẵn sàng. Khâu cuối ghi các nút, các **chunk** (mẩu chữ ngắn) và quyết định cho từng ứng viên tiêu đề, kiểm rằng bản thô chưa bị đọc lại trong lúc dựng, rồi ghi đầu mục của cây *sau cùng*; khi xoá thì đầu mục đi *trước*. Nhờ vậy, có đầu mục là có cả cây. Chỉ sau đó job mới gửi kết quả sẵn sàng về `ingest`.

7.3.4 Luật và chỗ canh luật

Bảng 7.10 ghi những luật `document` giữ, nơi thi hành chúng, và loại lưới canh chúng.

Bảng 7.10: Luật `document` giữ, nơi thi hành, và loại lưới canh chúng

Luật	Nơi thi hành	Loại lưới
Không giữ credential của database	cả service	`check_contract`
Mỗi thư viện bên ngoài chỉ được dùng thẳng ở đúng một tệp: object store, PDF, content store, Jev, mô hình ngôn ngữ	`storage`, `pdf`, `content_store`, `index/jev`, `index/llm`	`check_contract`
Kết quả sẵn sàng chỉ được gửi sau khi cây đã ghi	`index_handlers`	kiểm thử tự động
Đầu mục của cây ghi sau cùng, xoá đầu tiên	`content_store`	kiểm thử tự động
Đọc lại một tài liệu thì cây cũ bị xoá	`content_store`	kiểm thử tự động
Tệp hay trang không đọc nổi, cây không dựng được, kho hỏng tới lần thử cuối: đều thành một kết quả kèm lý do	`ocr_handlers`, `index_handlers`	kiểm thử tự động
Worker đọc tệp chạy một job một lúc	cấu hình worker	kiểm thử tự động
Không mở cổng nào	cấu hình chạy service	không gì canh

7.3.5 Khi hỏng

arq, thư viện chạy các worker, chỉ chạy lại một job khi job ấy chủ động xin chạy lại; một job chạm trần thời gian thì bị giết, và không ai báo gì. Vì vậy mọi cách hỏng của `document` phải tự biến thành một kết quả (bảng 7.11). Một tệp không đọc được hay không có chữ thì không nằm trong bảng này: đó là một kết quả bình thường của việc dò tệp. Khi giáo viên thấy lỗi, dòng trạng thái của tài liệu in “Xử lí lỗi. Hãy tải lại”, và lý do hiện khi trỏ chuột vào nó.

Bảng 7.11: Khi một thao tác bên trong `document` hỏng

Khi	`document` làm gì	Giáo viên thấy
Content store hay `redis` chập chờn	Xin chạy lại; tối đa năm lần chạy kể cả lần đầu, bốn lần chờ cộng lại 50 giây; lần thứ năm vẫn hỏng thì gửi kết quả hỏng	Dòng trạng thái vẫn báo đang xử lý, rồi báo lỗi
Một trang không đọc nổi giữa một lô	Gửi kết quả hỏng, nói rõ trang nào	Lỗi, kèm số trang
Một lô dài quá	Không mở trang mới sau 90 giây, dưới trần 120 giây, rồi giao phần còn lại cho lô kế	Không thấy gì
Job dựng cây chạy quá lâu	Tự cắt ở 540 giây, dưới trần 600 giây, rồi gửi kết quả hỏng kèm lý do	Lỗi
Bản thô bị đọc lại giữa lúc dựng cây	Lần đọc mới đã xong thì xin chạy lại trên bản mới; còn đang đọc thì dừng, không gửi gì, và lô OCR cuối sẽ giao lại job dựng cây	Không thấy gì
Không báo gì quá 300 giây: job bị giết, hay `redis` chết ở lần thử cuối	Lưới cuối nằm ngoài service: `backend` đọc tài liệu ra là hỏng	Lỗi

Tài liệu tham khảo

Báo cáo này mô tả một hệ thống đang chạy được, nên nguồn của nó là chính mã nguồn và các bản ghi quyết định đi kèm, chứ không phải tài liệu bên ngoài. Mọi con số và mọi luật nêu trong báo cáo đều có một chỗ tương ứng trong kho dưới đây.

Mã nguồn và tài liệu kỹ thuật của dự án
`https://github.com/biabeogo147/AI-Assessment-and-Feedback-Assistant`

Kho chứa năm **service** (thành phần) đã mô tả ở chương 6, mà ba trong số đó được đi vào bên trong ở chương 7, cùng ba nhóm tài liệu mà báo cáo này rút gọn từ đó:

- `docs/decisions/` — các bản ghi quyết định nghiệp vụ. Mỗi quyết định ở đó ghi cả phần *đã cân nhắc những gì* và *nơi nào thi hành nó*, hai phần mà báo cáo này không đưa vào.
- `docs/overview/` — mô tả nghiệp vụ, kiến trúc và mô hình dữ liệu. Nguồn của chương 4, 5, 6 và 7.
- `docs/diagrams/` — tệp gốc của mọi sơ đồ trong báo cáo. Các hình ở đây được *xuất ra* từ chúng chứ không vẽ lại, và một phép kiểm tự động trong kho sẽ báo đỏ nếu một sơ đồ đổi mà hình chưa xuất lại.

Các luật được canh bằng máy
`tools/check_contract.py` và `AGENTS.md` trong kho trên.